RightSize Bench: Learning Kubernetes Resource Requests from Code

Sizing a pod before it runs, submitted as a raw Job or through Flyte

Niels Bantilan
Union.ai
niels@union.ai

November 2026

Abstract

Before a batch pod runs on Kubernetes, someone has to decide how much memory and CPU it gets and whether it gets a GPU. Too little makes it fail or run slowly, and too much wastes the cluster. We present RightSize Bench, a benchmark that scores this decision in dollars: each request is replayed against a measured run, failed attempts and throttling included, and compared with the cheapest request that would have run it. The benchmark has 32,082 measured runs of 33 families of generated Python tasks that use real libraries, from dataframes and SQL to PyTorch training and sharded language models. Every run appears twice, as a raw Kubernetes Job whose inputs are staged in one of four ways, and as a task submitted through Flyte, an orchestrator whose runtime we measure at about 70 MiB per pod. We checked the Job labels in about 480 real pods, which found two errors in our first release.

We release the benchmark with a set of baseline models: gradient-boosted quantile trees, and a 1.7B decision model whose pointer heads score the request options written in its prompt, trained with cross-entropy and then with reinforcement learning against the cost. On new tasks from familiar families, the trees pooled with the decision model come within 11.7% of the cheapest request on Job pods, against 44.5% for a rule that scales earlier runs by input size and reads the pod spec, and 58.5% for a zero-shot 27B model. On seven held-out families every approach does much worse, and the rule (42.7%) is as good as anything we trained. Models trained on Jobs move their memory request with the volume medium in the manifest about three times in four. 81% of the test tasks share their exact code with a training task, so the held-out families are the benchmark's test of new code.

1 Introduction

Every container that runs on Kubernetes asks for resources before it starts [13], and each number fails differently. Memory is a hard limit: a container that uses more than it asked for is killed and has to run again. CPU is a soft limit: a container that wants more cores than it was given is throttled and finishes late. A GPU is a choice of machine. If the GPU is too small the task runs out of device memory, if a GPU task gets none it fails at start, and if a CPU task gets one it pays for hardware it never uses. Engineers usually pick these numbers by hand, once and generously, and large clusters run well below what their jobs reserve [21, 22].

Autoscalers that learn from usage history [15, 18] work well for long-running services. Batch work is harder, because the first run of a batch task has no history and later runs often see inputs of a different size. Much of what decides the right request is still known before the pod starts. A scheduler can read the code the pod will run, the size and format of its inputs, the way the pod spec stages those inputs, and what earlier runs of the same task used. This paper asks how well different approaches turn that information into a request, and measures the answer in dollars.

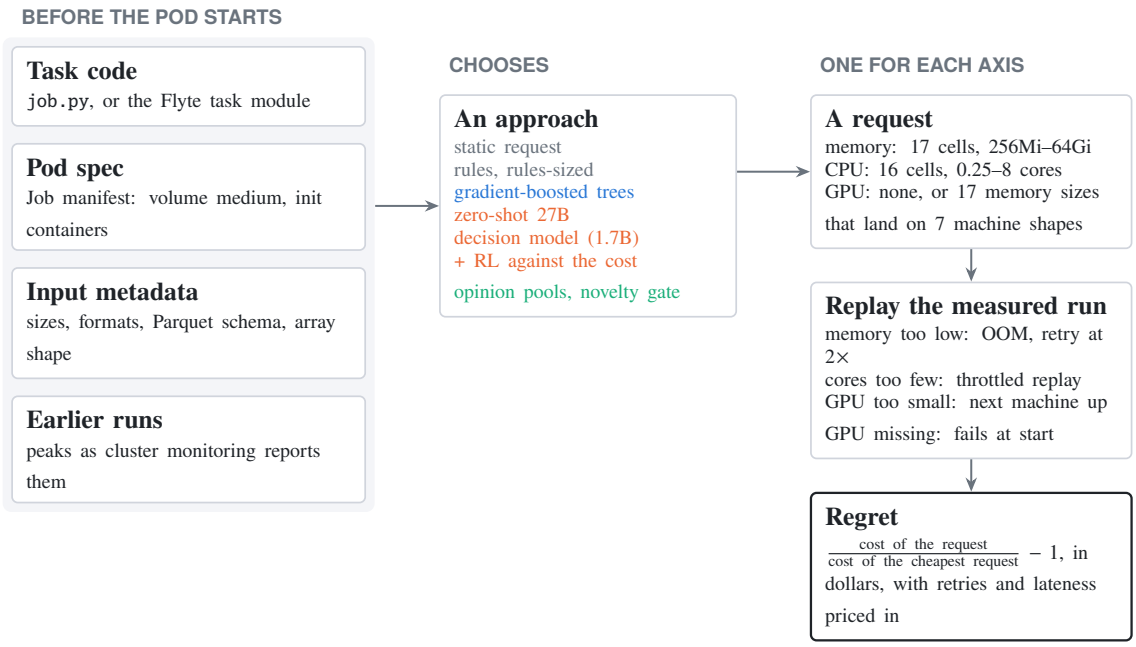


Figure 1: What RightSize Bench measures. An approach reads what a scheduler can see before a pod starts and chooses a request on each axis. The benchmark replays the measured run under that request and prices the result, failed attempts and throttling included, against the cheapest request that would have run it.

We treat the Kubernetes pod as the unit of work. Teams submit pods in different ways. The most direct is a `batch/v1` Job, a container image paired with a manifest. Another is to write a typed Python function and let an orchestrator build the pod. We use Flyte [5] as the example of the second route: it turns a function into a pod and adds typed inputs and outputs, caching, retries and durable execution. In both cases the same user code runs in the same kind of pod. What differs is how inputs arrive, what else shares the container, and what the monitoring history reports.

The contributions are a benchmark and a first set of models trained on it (Figure 1).

1. **RightSize Bench.** 32,082 measured runs of 33 families of generated Python tasks covering dataframes, SQL, linear algebra, classical ML, PyTorch training and inference, and sharded language models, with measured peak memory, CPU use at the 15-second resolution of cluster monitoring, and peak GPU memory across seven machine shapes (Section 5). Every run is presented as a raw Kubernetes Job, with one of four ways of staging its inputs, and as a Flyte task: 64,164 pods in all (Section 6).
2. **Labels checked in real pods.** We checked the derived Job labels in about 480 pods, reading each container's own cgroup counters (Section 7). The check found two errors in our first release: the Flyte runtime constant was 2.6 times too large, and some init containers were too small to hold the inputs they staged.
3. **A cost-based evaluation.** One cost model covers memory, CPU and GPU together, and regret is measured against the cheapest request under that model (Section 8). Results carry intervals clustered by task and, for held-out families, by family; each approach's read-out is chosen on validation; and every trained approach is trained three times.
4. **Baseline models.** Gradient-boosted quantile trees over 209 scheduler-visible features, and a decision model that puts three pointer heads on Qwen3-1.7B so that one forward pass scores every option written in

the prompt, trained with cross-entropy and then with reinforcement learning against the cost (Section 9). We compare them with static requests, two rules, a zero-shot 27B model, opinion pools and a novelty gate (Section 11).

5. **An open release.** The dataset, with prompts, features and every approach's requests, and the trained models, with standalone inference code, are on the Hugging Face Hub, and the benchmark itself is a Flyte factory that rebuilds only what changed (Section 12).

The models trained on Jobs read the pod spec: on first runs where only the volume medium changes, they move their memory request the right way about three times in four. On familiar code the best of them come within 12–14% of the cheapest request. On held-out families, a hand-written rule that scales earlier runs by input size and reads the manifest does as well as any of them.

2 Background: one pod, two ways to submit it

Requests, limits and cgroups. Each Kubernetes container declares a memory and CPU *request*, which the scheduler reserves on a node, and a *limit*, which the kubelet enforces through the container's cgroup [9, 13]. Under cgroup v2, a container whose charged memory reaches its limit is OOM-killed. Anonymous memory and shared memory (`shmem`) both count toward the limit. Clean page cache does not, because the kernel can reclaim it first. A CPU limit is enforced as a CFS quota, so a container that exceeds it is slowed down instead of killed. Accelerators are requested as whole devices on node pools that carry them. In this paper a request also sets the limit, the usual choice for batch pods because it makes scheduling and failures predictable.

How the pod spec changes a task's memory. Three details of a pod spec change how much memory a batch task needs without any change to its code. The first is the medium of the volume that holds its inputs. An `emptyDir` on node disk keeps files as page cache, which the kernel reclaims under pressure. An `emptyDir` with `medium: Memory` is a tmpfs, and every byte written to it is charged as `shmem` [14], so staging a 2 GiB input on tmpfs adds 2 GiB to the pod's need. The second is who fetches the inputs. An init container runs to completion before the main container starts and has its own limit, while a download inside the job runs in the job's container, so the client library's memory counts against the job. The third is anything else that shares the container, such as an orchestration runtime.

A tmpfs page is charged to the cgroup of the container that writes it, so pages an init container writes are not charged to the main container. When the init container exits, its pages move to the pod's cgroup and still count against the pod's limit. We confirm this in real pods in Section 7.

Raw Jobs. A `batch/v1` Job runs a pod template until it succeeds. Its `backoffLimit` says how many failed pods it replaces before giving up; batch teams often set it to zero and resubmit failures by hand or from an outer scheduler. A Job stages its own inputs, with an init container or in the job's code.

Flyte. Flyte [5] turns a typed Python function into a Kubernetes pod. A user writes `@env.task async def f(x: File) -> dict`, and Flyte builds the image, renders the pod spec, moves typed inputs and outputs through object storage, caches results by their inputs, retries failed attempts, and records each execution so that a long pipeline can resume after a node is lost. None of this changes what the function does inside the container. It changes the pod's edges: inputs arrive through Flyte's download, the container also runs Flyte's runtime, and the execution history reports the container's peak with that runtime included. We measure the runtime at 55 MiB at rest and a median 70 MiB above the same workload run as a plain Job (Section 7). An approach that can size a Kubernetes pod should therefore be able to size a Flyte task with the same knowledge plus the runtime.

3 Related work

Sizing from history. Autopilot [18] sets Google's container limits from the usage history of a job's tasks, using moving-window and ML recommenders. Kubernetes' Vertical Pod Autoscaler [15] recommends requests from a decaying histogram of observed usage. Borg traces show that most jobs over-request [21, 22]. Resource Central [1] predicts VM utilization from a subscription's history. All of these size workloads that have run before, and run long enough to observe. RightSize Bench reports first runs (no history) and runs with history separately. Quasar [2] classifies new workloads from short profiling runs; RightSize Bench allows no profiling, only what a scheduler can read before the pod starts.

Learned predictors. Quantile regression [12] with gradient-boosted trees [6, 11] is a standard way to predict a distribution of resource need; it is the strongest classical approach here. Our decision model replaces a language model's output layer with a pointer head [23] that scores option tokens written in the prompt. It is adapted with LoRA [10] on Qwen3 [17].

Training on the objective. Group-relative policy optimization (GRPO) [19] drops PPO's value network by normalizing rewards within a group of samples for the same prompt. We use that group-normalized advantage with a single on-policy step per batch. Our policy is three categorical distributions, so a group is sampled from one forward pass, and the reward is the benchmark's own cost simulator.

Combining and routing models. Logarithmic opinion pools [7] combine calibrated distributions by a weighted geometric mean. Temperature scaling [8] calibrates each model before pooling. Distance to the k nearest training points is a common out-of-distribution signal [20]; our gate uses it to choose between two models.

4 Problem

A *run* is one execution of a task on one set of inputs, in one pod. Before the pod starts, an approach may look at three things. The first is the task's code as it was submitted: `job.py` and the Job manifest for a raw Job, and the task module for a Flyte task (Section 6). The second is the metadata a scheduler can read about this run's inputs: their sizes and formats and whatever the format's header exposes, such as the row count and schema of a Parquet file or the shape and dtype of a `.npy` array. The third is what earlier runs of the same task used: their peak memory, peak and mean cores, peak GPU memory and utilization, wall time and input sizes. These are reported the way the platform's monitoring would show them, which means the container's own peak for a Job and the peak including the runtime for a Flyte task.

An approach is never given the task's family, the hidden settings it was generated with, the corpus it came from, or anything measured on the run itself. A unit test checks that changing those fields leaves the features and the prompt unchanged. The code does identify the family in practice: each task is named after its family, and each family's generator has a fixed function name and input schema. Section 11.4 re-scores with neutral names.

From what it can see, the approach chooses a request on three axes (Table 1). A GPU request lands on the cheapest machine that can hold it (Table 2). On a GPU node the host memory and cores come with the instance, so they are not scored.

Table 1: The request grid.

axis	cells	values
memory	17	256, 384, 512, 768 Mi; 1, 1.5, 2, 3, 4, 6, 8, 12, 16, 24, 32, 48, 64 Gi (half powers of two)
CPU	16	0.25, 0.5, 0.75, 1, 1.25, 1.5, 1.75, 2, 2.5, 3, 3.5, 4, 5, 6, 7, 8 cores
GPU	18	none, or peak GPU memory 1, 1.5, 2, 3, 4, 6, 8, 10, 12, 15, 22, 32, 44, 64, 88, 120, 176 GiB

Table 2: The GPU machine ladder: the cluster's GPU node pools, with on-demand `us-east-1` prices for the whole node. Usable GiB is NVML's total, rounded down. The A10G is never the cheapest shape for any cell.

shape	instance	GPUs	usable GiB	$/h
T4	g4dn.xlarge	1	15	0.526
L4	g6.2xlarge	1	22	0.978
A10G	g5.2xlarge	1	22	1.212
L40S	g6e.2xlarge	1	44	2.242
4×L4	g6.12xlarge	4	4×22	4.602
8×T4	g4dn.metal	8	8×15	7.824
4×L40S	g6e.12xlarge	4	4×44	10.493

5 Dataset

The benchmark combines three corpora, each built by its own single-axis benchmark with the same recipe. We wrote families of code generators, sampled tasks from each family, and measured several runs of each task at different input sizes in Kubernetes pods. Table 3 summarizes them, Figure 2 shows the range of labels they cover, and Appendix A lists every family. These measurements describe the workload itself. Section 6 works out what a Job pod and a Flyte pod each need from the same measurement.

5.1 Families, tasks and runs

A *family* is a Python generator. Given a seed, it writes a task function and the specification of its inputs, and it draws a few hidden settings, which we call knobs, that decide how input bytes turn into resource use. A pandas family decides whether to project columns, which dtype backend to use and whether to downcast floats. A training family picks width, depth, batch size, mixed precision and checkpointing, and a CPU family picks a thread-pool size or leaves it unset. The knobs show up only in the code an approach reads; we keep them to slice results.

Several families have small knob spaces, so their tasks repeat. In RightSizeMem, `python_grouping`

Table 3: The three corpora. RightSizeGPU's 2,052 unusable runs are 1,569 censored runs (out of memory on the largest machine they reached), 51 failures, and 432 runs whose peak filled the device, which makes the measured peak only a lower bound. Most censoring came from cluster capacity: every 4×L4 shard and half of the 8×T4 shards timed out in the queue, so runs that needed those machines never reached them. RightSizeMem's 166 unusable runs exceeded its 20 GiB guard or its 900 s timeout.

corpus	release	families	held out	tasks/fam.	runs/task	measured	usable	labels
RightSizeMem	v2	12	2	400	3	14,400	14,234	peak memory; mean cores
RightSizeCPU	big-1	11	2	450	2	9,900	9,900	peak memory; busiest 15 s
RightSizeGPU	big-2	10	3	500	2	10,000	7,948	peak GPU memory, sharding
combined	k8s-2	33	7			34,300	32,082	

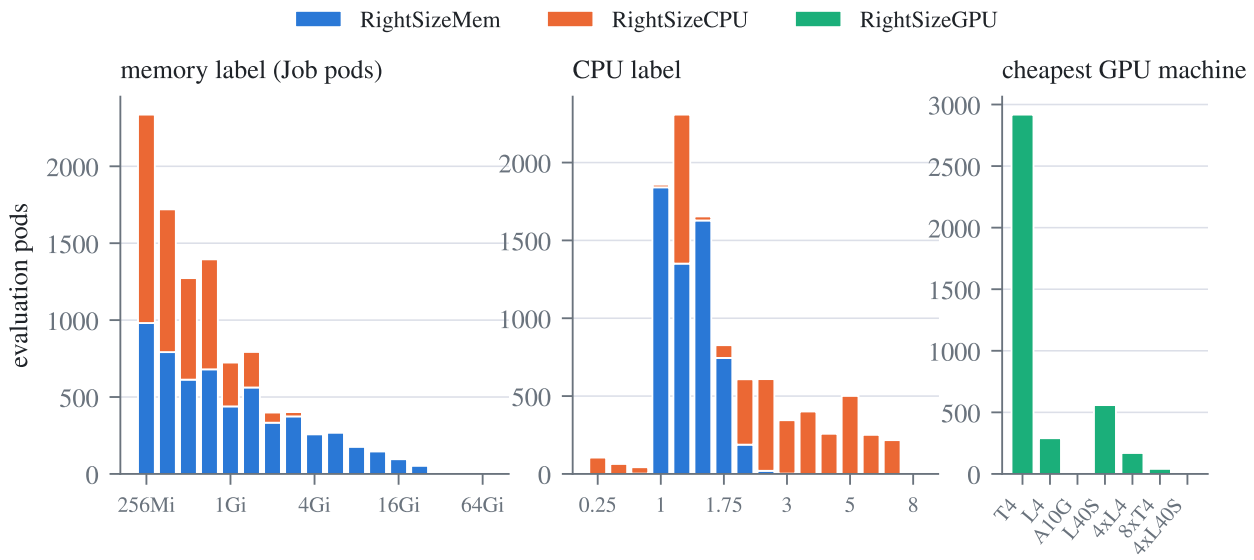


Figure 2: The labels of the 28,058 evaluation pods (validation, test and unseen), Job pods shown. Memory spans 256 Mi to 24 Gi, with RightSizeMem reaching the larger cells. CPU spans 0.25 to 8 cores; RightSizeMem's CPU label is mean cores and RightSizeCPU's is the busiest 15 seconds. A GPU label is the cheapest machine the run's peak fits; 4×L4 labels come from the fit model, because no 4×L4 shard ran.

has 3 distinct programs across its 400 tasks and `jsonl_records` has 4, while `torch_train` has 327. Of the 5,095 test Job pods, 4,135 (81%) run code identical to some training task's, and differ from it only in input sizes and history. The remaining 960 test pods are new programs from familiar families, and the seven held-out families are new kinds of code. Section 11.2 reports all three.

Each run scales the leading dimension of the task's inputs by a factor drawn log-uniformly from $[0.25, 4]$ for RightSizeMem and from $[0.5, 2]$ for the CPU and GPU corpora. Runs are shuffled, so a task's history can be larger or smaller than the current run. The inputs are synthetic (normal floats, uniform integers, dictionary-encoded categories, random strings, Zipf-distributed word ids, and random bytes for images), written as Parquet, CSV, JSONL or `.npy`. CPU and GPU workloads loop until a per-task time budget runs out (30–60 s for CPU and 45–90 s for GPU), while RightSizeMem workloads run to completion.

5.2 RightSizeMem v2: peak memory

RightSizeMem has twelve families, chosen because they turn input bytes into memory in different ways: pandas group-bys and joins, polars pipelines in eager, lazy and streaming modes, DuckDB queries with and without a memory limit, chunked CSV and JSONL readers, NumPy linear algebra with optional memory mapping, scikit-learn estimators, TF-IDF, a small PyTorch model on CPU, image batches and pure-Python grouping.

Each run uses three processes. A child process writes the inputs to a temporary directory, a fresh interpreter runs the task body on those files, and the parent reads the child's `ru_maxrss` and CPU time from `wait4`. BLAS and OpenMP are limited to one thread, and workloads set their own threads in code. The workload's peak memory w is its `ru_maxrss`. On Linux this value includes the measuring parent's own peak at the time of `exec`, about 158 MiB here, which raises about a third of the smaller workloads' labels by one cell (Section 7).

Table 4: Usable runs per split. Each run appears twice, as a Job and as a Flyte task, in the same split.

source	train	val	test	unseen	unseen families
mem	8,457	1,138	2,316	2,323	duckdb_sql, image_batch
cpu	5,628	838	1,634	1,800	duckdb_sql, pyarrow_csv
gpu	3,968	542	1,145	2,293	recsys_train, knn_search, llm_generate
runs	18,053	2,518	5,095	6,416	
pods (rows)	36,106	5,036	10,190	12,832	

5.3 RightSizeCPU big-1: the busiest 15 seconds

RightSizeCPU has eleven families, chosen for how they use cores, since CPU need depends on parallelism more than on input size. They cover NumPy BLAS with and without a thread cap, polars and DuckDB with explicit thread settings, scikit-learn ensembles, serial pandas, Python thread pools that hold or release the GIL, process pools, a CPU PyTorch model, pyarrow CSV parsing, an I/O-bound pool and a task with two phases.

CPU time for the whole process tree is sampled every second. Each pod runs two measurements side by side on disjoint sets of seven cores; a check of 44 runs against single-run pods put all 44 within 25% of each other. The CPU label is the busiest 15-second average, because Prometheus scraping cAdvisor every 15 s sees a container at that resolution. Peak memory is measured as in RightSizeMem.

5.4 RightSizeGPU big-2: peak GPU memory across a machine ladder

RightSizeGPU has ten PyTorch families: a tabular MLP, a CNN, a CNN limited by its data loader, a GPT language model, a transformer encoder, a recommender with sharded embeddings, blocked matrix multiplication, nearest-neighbour search, fine-tuning of decoders from 0.6B to 7B parameters split across GPUs, and generation with a KV cache. NVML reports memory and utilization on every visible GPU every half second. Each run starts on a T4 and moves up the machine ladder (Table 2) whenever it runs out of device memory, and a run that runs out of memory on the largest machine it reached is censored. The label is the cheapest machine the run fits on. For machines a run was not measured on, we model the fit from its per-GPU peak and, for sharded code, its imbalance across GPUs. Single-GPU runs that outgrew a T4 were measured on an L40S, because L4 capacity was not available, and their L4 labels come from that peak.

5.5 Splits and history

Runs are split by task, never by run. A hash of the task id sends 70% of each family's tasks to training, 10% to validation and 20% to test. Seven held-out families go entirely to a separate *unseen* split: DuckDB SQL (in both RightSizeMem and RightSizeCPU), image batches, pyarrow CSV parsing, nearest-neighbour search, generation with a KV cache, and the sharded recommender. Only DuckDB is a library no training family imports; the other five use libraries the training families also use, in code the training families do not contain. Every setting an approach tunes is chosen on validation, which has familiar families only.

The k-th run of a task carries the task's earlier runs as its history. A task's first run has no history, and these cold starts make up 33–50% of each split (Table 4).

6 Every run as a Job and as a Flyte task

The workload code was generated as Flyte tasks, and nothing in the body of a task depends on Flyte. We use that to put every measured run into the benchmark twice, once as a raw Kubernetes Job and once as a

Table 5: The four Job stagings.

staging	share	who fetches the inputs	where they land	memory need beyond the workload
init-disk	30%	aws-cli init container	`emptyDir` on node disk	—
init-tmpfs	20%	aws-cli init container	`emptyDir`, `medium: Memory`	$+B$ (staged bytes)
boto3-disk	30%	the job itself, boto3	`emptyDir` on node disk	+48 MiB (boto3)
boto3-tmpfs	20%	the job itself, boto3	`emptyDir`, `medium: Memory`	+48 MiB $+B$

Flyte task, with the task body unchanged in both. To make the Job version, `job.py`, we turn `File` parameters into paths, drop the `flyte` import, and add a `__main__` block that stages the inputs, calls the function and writes `result.json` for upload. A `batch/v1` manifest goes with it, carrying the image, the input locations, the thread settings the measurement used, the staging volume and an empty `resources` block, which is the request still to be decided. Figure 3 shows one task written both ways.

6.1 Stagings

How a Job's inputs reach its container is the biggest thing that changes its memory need. Each task is assigned one of four stagings by a hash of its id, and all of its runs share it (Table 5). Inputs are fetched either by an init container running `aws-cli` or by the job itself with boto3, and they land either on node disk or on a memory-backed volume.

6.2 Label accounting

Let w be the workload's measured peak memory (Section 5) and let B be the size of the staged inputs plus the result file, with each file rounded up to whole pages. The two kinds of pod need

$$\begin{aligned} \text{Job:} \quad & 1.05\,\big(w + 48\,\text{MiB} \cdot [\text{boto3}] + B \cdot [\text{tmpfs}]\big), \\ \text{Flyte:} \quad & 1.05\,\big(w + R\big), \qquad R = 70\,\text{MiB}, \end{aligned}$$

where the factor 1.05 is headroom and the memory label is the smallest grid cell at or above the need. Files on a memory-backed volume are charged to memory byte for byte, while the same files on node disk are reclaimable page cache and add nothing, since the measured workloads already read local files. The boto3 client adds 48 MiB to a workload's peak. Its download also runs inside the job, taking $d = \text{bytes}/440\,\text{MB/s} + 0.4\,\text{s}$ per object $+ 0.7\,\text{s}$ at about 1.8 cores, so we prepend that window to the run's CPU series. For a light job with a large input, the download can be the busiest 15 seconds of the run. The Flyte runtime R is 70 MiB, measured in real task pods (Section 7). Our first release used 180 MiB, a value no probe had measured. Staging does not change GPU need.

The manifest sizes the init container at 256 MiB + B for tmpfs stagings, so that it can hold what it writes. Our first release used a flat 256 Mi, which real pods showed cannot work for larger inputs (Section 7). Histories are reported in each interface's own units: a Job's earlier runs report the container's peak as cAdvisor would, and a Flyte task's report the peak including the runtime, as Flyte's UI would. A Job's history is computed with the same accounting as its label, so an earlier run's reported peak is exactly its need without the headroom. Real cAdvisor peaks are noisier, and `max_usage` also counts page cache. The init container's own download time is not priced, while the boto3 stagings pay for theirs inside the job, which makes the init stagings slightly cheaper than they are.

The staging constants come from an in-cluster probe using boto3 1.43 under cgroup v2. A bare interpreter holds 11 MiB. With boto3 imported, a client built and a file downloaded, the process holds 74–80 MiB, and it never peaks above that final level. Inside a real workload boto3 adds only 48 MiB at the median, because it

Flyte task

```
import flyte
from flyte.io import File
import pandas as pd

env = flyte.TaskEnvironment(name="pandas-join",
     image=flyte.Image.from_debian_base().
     with_pip_packages("pandas", "pyarrow"))


@env.task
async def join_orders_catalog(orders: File, catalog:
      File) -> dict:
    orders_path = await orders.download()
    catalog_path = await catalog.download()
    orders = pd.read_parquet(orders_path)
    catalog = pd.read_parquet(catalog_path)
    joined = orders.merge(catalog, on="sku", how="
     inner")
    joined["revenue"] = joined["qty"] * joined["
     price"]
    by_brand = joined.groupby("brand", observed=True
     )["revenue"].sum()
    return {"rows": int(len(joined)), "brands": int(
     len(by_brand))}
```

Job manifest (boto3-tmpfs)

```
apiVersion: batch/v1
kind: Job
metadata:
  generateName: pandas-join-
  labels: {app.kubernetes.io/name: pandas-join}
spec:
  backoffLimit: 0
  template:
    spec:
      restartPolicy: Never
      containers:
        - name: main
          image: registry.internal/batch/pandas-join
    :3ee28a49
          command: [python, /app/job.py]
          env:
            - {name: INPUT_ORDERS, value: "s3://
    batch-data/pandas-join/a1b2c3d4/run-1/orders.
    parquet"}
            - {name: INPUT_CATALOG, value: "s3://
    batch-data/pandas-join/a1b2c3d4/run-1/catalog.
    parquet"}
            - {name: OMP_NUM_THREADS, value: "1"}
            - {name: OPENBLAS_NUM_THREADS, value:
    "1"}
            - {name: MKL_NUM_THREADS, value: "1"}
            - {name: OUTPUT_URI, value: "s3://batch-
    data/pandas-join/a1b2c3d4/run-1/out/"}
          resources: {}  # to be decided
          volumeMounts: [{name: data, mountPath: /
    data}]
      volumes:
        - name: data
          emptyDir: {medium: Memory}
```

`job.py` (boto3 staging)

```
import pandas as pd
import json
import os
import boto3


def join_orders_catalog(orders_path: str, catalog_path:
      str) -> dict:
    orders = pd.read_parquet(orders_path)
    catalog = pd.read_parquet(catalog_path)
    joined = orders.merge(catalog, on="sku", how="inner
     ")
    joined["revenue"] = joined["qty"] * joined["price"
     ]
    by_brand = joined.groupby("brand", observed=True)[
     "revenue"].sum()
    return {"rows": int(len(joined)), "brands": int(
     len(by_brand))}


def fetch(uri: str) -> str:
    bucket, key = uri.removeprefix("s3://").split("/",
      1)
    path = os.path.join("/data", os.path.basename(key))

    s3.download_file(bucket, key, path)
    return path


if __name__ == "__main__":
    s3 = boto3.client("s3")
    result = join_orders_catalog(fetch(os.environ["
     INPUT_ORDERS"]), fetch(os.environ["INPUT_CATALOG"
     ]))
    os.makedirs("/data/out", exist_ok=True)
    with open("/data/out/result.json", "w") as f:
        json.dump(result, f)
    bucket, key = os.environ["OUTPUT_URI"].removeprefix
     ("s3://").split("/", 1)
    s3.upload_file("/data/out/result.json", bucket, key
      + "result.json")
```

Figure 3: One RightSizeMem task, as a Flyte task and as a raw Job. The disk staging differs from this manifest in one line, `emptyDir: {}`. The init-container stagings add an `aws-cli` init container that copies the inputs to `/data`, and `job.py` then reads them from `argv` (Appendix B).

shares modules the workload has already loaded. Download throughput rises from 122 MB/s for a 64 MiB object to 417–431 MB/s for a 4 GiB one.

7 Checking the labels in real pods

The Job labels in Section 6.2 are derived from the workload's measurement, so we checked them twice. The first check (rt-k8sval-2) re-ran 176 corpus runs both as Flyte tasks and as their rendered `job.py`, as sibling processes inside one pod. It confirmed the arithmetic of the accounting. Measured over predicted memory was 0.999–1.000 at the median for every staging, and tmpfs was charged at exactly the input size. A pod with one container has one memory cgroup, so this check could not show where the kernel charges pages when a pod has several containers.

The second check (rt-k8spod-1) used real pods. We took 100 runs (48 from RightSizeMem, 44 from RightSizeCPU, and 8 init-tmpfs runs with inputs over 300 MiB) and gave each of them pods of its own, about 480 in all (Table 6). Every container sampled its own cgroup's memory counters every 50 milliseconds.

The Flyte runtime. Before a task's own module is imported, the Flyte task container already holds 55.0 MiB of anonymous memory, with almost no variation across the 100 runs (p10–p90 54.9–55.0 MiB). Its peak is a median 70 MiB above the same run's peak as a plain Job (p10 53, p90 242 MiB), so running a workload inside the runtime's process costs slightly more than the runtime does on its own. With the first release's 180 MiB, the Flyte label matched the real container's cell for only 23% of runs and was one cell too high for 70%. With 70 MiB, it matches for 58%, is one cell high for 33% and one cell low for 9%. The low ones are process pools, where the container counts every worker process, and some DuckDB and polars runs. The k8s-2 release uses 70 MiB, which lowers 61% of the Flyte labels on CPU nodes by one cell.

Where tmpfs pages are charged. A tmpfs page is charged to the container that writes it. When the job downloads its own inputs (boto3-tmpfs), the job container's shared memory grows by the staged size times 1.0001 at the median ($n = 23$). When an init container stages them (init-tmpfs), the init container's shared memory equals the staged size and the job container's stays at zero ($n = 31$). The pages outlive the init container, however, and still count against the pod. With its limit set to the label minus the staged bytes, the job was OOM-killed in 9 of 31 init-tmpfs pods, and with the full label 30 of 31 ran. Adding the staged bytes to the label is therefore correct; the pages are enforced through the pod's limit, not the container's.

The first release's init containers. With the flat 256 Mi init container of the first release, every staging up to 162 MiB ran and none of 254 MiB or more did. Ten of those were confirmed OOM-killed while staging, and six more failed. In that release, 982 of the 2,756 evaluated init-tmpfs Jobs staged more than 200 MiB, which means 7% of all Job rows described pods that could never have started. The k8s-2 release sizes the init container at 256 MiB plus the staged bytes.

Table 6: The pods in the real-pod check (rt-k8spod-1).

pod	what it measures
Flyte task	the run as a real Flyte task: the rendered module in a fresh task pod, with its inputs downloaded by Flyte
Job, generous limit	the pod the manifest describes, with each container reading its own cgroup
Job at its label	does the run fit when the job container's limit is set to the label?
Job one cell lower	does the cell below the label fail?
init-tmpfs extras	the first release's flat 256 Mi init container, and the job at its label minus the staged bytes

Table 7: Job pods with the limit at the label cell and one cell below it (rt-k8spod-1). "Other" is a failure that was not an OOM kill.

	at label			one cell below		
staging	ran	OOM	other	ran	OOM	other
init-disk	22	1	0	17	5	1
init-tmpfs	30	1	0	24	7	0
boto3-disk	23	0	0	16	6	1
boto3-tmpfs	23	0	0	15	7	1

Table 8: The corpus Job label compared with the job container's measured cgroup peak, in grid cells.

staging	n	same cell	within one	higher	lower
init-disk	23	65%	91%	30%	4%
init-tmpfs	31	68%	97%	26%	6%
boto3-disk	23	52%	96%	48%	0%
boto3-tmpfs	23	52%	100%	48%	0%

Labels as limits. When we set the job container's limit to its label, 98 of 100 runs succeeded (Table 7). Both failures were process pools, where the label records the largest single process but the cgroup counts all of them together. One cell below the label, 25 of the 97 runs that started were OOM-killed.

Labels are sometimes a cell too generous. We also compared each corpus label with the job container's measured peak of anonymous plus shared memory, plus the same 5% headroom (Table 8). Depending on the staging, the label falls in the same cell for 52–68% of runs and within one cell for 91–100%. It is higher for 26–48% of runs and lower for at most 6%. Two things push labels up. The first is a quirk of `ru_maxrss`: on Linux, the value a parent reads for its child through `wait4` includes the parent's own high-water mark at the moment the child calls `exec`, because the kernel carries the old address space's peak into the new one. The measuring parent in our corpora held about 158 MiB, so even the smallest workloads report at least that much. The second is memory-mapped input files, which RSS counts but the kernel can reclaim; `numpy_linalg` and `image_batch` both map their inputs. Neither effect makes a run fail at its label, and together they make about a third of labels one cell larger than necessary. The same comparison explains why 72 of the 97 Job pods that started one cell below their label still ran: for many of them the label is one cell high. The simulator of Section 8 replays the measured need, so the cheapest request it finds inherits the same bias, and regret is measured against a slightly generous floor.

8 Cost model and metrics

We score a request by what it costs to get the pod to finish, counting failed attempts, retries and lateness, all simulated from the run's measurements. Each axis fails the way it fails on Kubernetes. If memory is too low, the container is OOM-killed when the run reaches its peak (at the measured fraction of its runtime) and is retried with double the memory, for up to eight attempts. If CPU is too low, the run is throttled: each second that wanted u cores takes $\max(1, u/\text{limit})$ seconds, replayed against the measured per-second series. If the GPU is too small, the run hits a CUDA out-of-memory error and is retried on the next machine up, for up to four attempts. A GPU task given no GPU fails at start and is retried on a T4. A CPU task given a GPU runs normally and pays the GPU node's price.

A CPU pod is priced by its dominant share of an `m6i.xlarge` (4 vCPU, 16 GiB, $0.192 per hour), that is $p(m, c) = 0.192 \cdot \max(c/4,\ \mathrm{GiB}(m)/16)$, and a GPU pod pays the on-demand price of its machine. Every attempt pays a start-up overhead: 0.02 hours on a CPU node and 0.05 hours on a GPU node, for image pull, device initialization and reading inputs. We normalize every run to one nominal hour of work, so that short measured runs stand in for longer production tasks. A run that exhausts its attempts pays for the attempts it made; this happens for fewer than 0.4% of evaluated pods for any approach.

Two penalties price time. Each retry adds the cost of running the label request once, which stands for the delay of running again, and each hour of lateness from throttling adds the same amount per hour. The label request's cost is a fixed quantity for each run, so the penalties do not depend on the request being scored. These defaults match an orchestrator that retries automatically. A raw Job with `backoffLimit: 0` fails and waits for someone to resubmit it, so Section 11.7 also scores every approach with retry penalties of 0, 5 and 20. Because costs include these penalties, the "dollars" in this paper are priced dollars, not a cloud bill.

The cheapest request. For each pod, the benchmark searches the grid for the request with the lowest cost under this model: every memory and CPU cell with no GPU for a CPU task (272 requests), and every GPU cell for a GPU task. This is the denominator of regret, and no request can beat it. It is often cheaper than the label, the smallest cell that fits: asking for one CPU cell below the busiest 15 seconds only throttles that window, and the cores saved over the rest of the run can cost less than the throttling. Against the label, our first release reported negative regret for some approaches on RightSizeCPU; against the cheapest request, regret is never negative.

Metrics. The headline metric is *regret*, the mean over pods of an approach's cost divided by the cheapest request's cost, minus one. Every pod counts equally however much it costs, and this is the quantity every approach tunes on validation. *Dollar regret* is total cost over total cheapest cost, minus one, so large jobs count for more. We also report the share of runs whose first attempt succeeds. For pairs of pods that come from the same run but differ in one detail of the pod spec, we check whether an approach's memory request moves in the same direction as the label (Section 11.4), separately for pairs whose label moved and pairs whose label did not.

Uncertainty. Runs of the same task share code, and a run's Job and Flyte pods share a workload, so pods are not independent. Intervals are 95% percentile intervals from a clustered Poisson bootstrap [3, 4] with 2,000 replicates. By default each task, with all of its runs and both pods of each run, gets one Poisson(1) weight; this measures uncertainty about new tasks from the same families. A claim about new kinds of code has only seven held-out families behind it, so for the unseen split we also cluster by family, which gives much wider intervals. Differences between approaches and between pod types are bootstrapped as paired differences on the same pods. Variation between trainings is reported separately, across three seeds (Section 11.8).

9 Approaches

Every approach produces a full request for every pod. Settings that an approach tunes, including which of its read-outs to report, are chosen on the validation split by mean regret; the reinforcement-learning settings are fixed and listed with the rest in Appendix C.

9.1 Static requests and rules

Static gives every pod the same request: a training quantile of memory need and of CPU need, and no GPU, so every GPU task fails at start and is retried on a T4.

Rules are what a platform team might write by hand. If the code mentions CUDA, the task gets a GPU sized from its largest past GPU peak times a margin. Memory is the largest past peak, converted to this pod's units and multiplied by a margin, or the static request if there is no history. CPU is the largest past peak in cores times a margin; on a first run it falls back to the largest pool size the code declares (`max_workers=`, `n_jobs=` and so on), or one core.

Rules-sized also reads input sizes and the pod spec. With history, it takes the earlier run closest in input size and scales its peak by the ratio of input sizes. On a first run it estimates the workload's peak as $a + k \cdot$ input MiB, with a and k tuned on validation first runs, and adds the staging's cost as read from the pod spec: 48 MiB if `job.py` imports boto3, and the staged bytes if the manifest says `medium: Memory`. It then converts that peak into the pod's need with the accounting of Section 6.2. Rules-sized therefore uses the benchmark's own accounting constants (the runtime, the boto3 footprint, the headroom), which a learned model has to infer from data.

9.2 Gradient-boosted quantile trees

The trees use 209 features a scheduler can compute. Twenty-seven describe the inputs and history: total bytes, bytes by format, visible rows, column counts, array shapes, the number of earlier runs, their peaks, and the size ratio to the closest earlier run. Another 173 mark API tokens in the code, such as `groupby`, `TfidfVectorizer` or `ProcessPoolExecutor`, and numeric literals such as pool sizes, layer widths and batch sizes. Four flag CUDA use, and five flag details of a Job's pod spec (`kind: Job`, `initContainers`, `medium: Memory`, `import boto3` and `download_file`).

For each axis we fit 19 quantile regressors [12] with scikit-learn's `HistGradientBoostingRegressor` (quantiles 0.05 to 0.95, 300 rounds, learning rate 0.06, 31 leaves) on the base-2 log of the need, plus a classifier that predicts whether the task needs a GPU. The predicted quantiles define a piecewise-linear CDF, and each cell's probability is the CDF mass between its edges. The fit is deterministic. Two ablations train on Job pods only or on Flyte pods only.

9.3 Zero-shot 27B

A 27B-parameter Qwen instruction model, served with vLLM [16], reads the prompt shown in Appendix B and answers in JSON at temperature zero. We read units as Kubernetes would (so `2 GiB`, `2GB` and `500m` all parse) and round each value up onto the grid; an answer we cannot parse falls back to 4 Gi, one core and no GPU. The model writes plain-text reasoning before its answer even with thinking turned off, so we give it up to 8,192 tokens, and we also score a variant whose decoding is constrained to the answer's JSON schema with 256 tokens. Both are scored on a family-stratified sample of 150 runs per split, each run's Job and Flyte pods, which leaves 130 test and 150 unseen Job pods. Section 11.9 compares every approach on that same sample.

9.4 Decision model: three pointer heads on one forward pass

The decision model reads the prompt and scores every option on every axis in one forward pass (Figure 4). The options are written out at the end of the prompt. For each axis, a small head compares the hidden state at the prompt's last token with the hidden state at each option's last token:

$$s_{\text{axis}}(j) = \frac{\langle W_{q,\text{axis}}\,\text{LN}(a),\ W_{k,\text{axis}}\,\text{LN}(k_j)\rangle}{\sqrt{256}}, \qquad P(\text{axis} = j) = \text{softmax}_j\big(s_{\text{axis}}(j)/T_{\text{axis}}\big). \tag{1}$$

Here a is the last prompt token's hidden state, k_j is option j's, each axis has its own affine maps W_q, W_k from 2,048 to 256 dimensions, LN is one shared LayerNorm, and the temperature T_{axis} minimizes validation

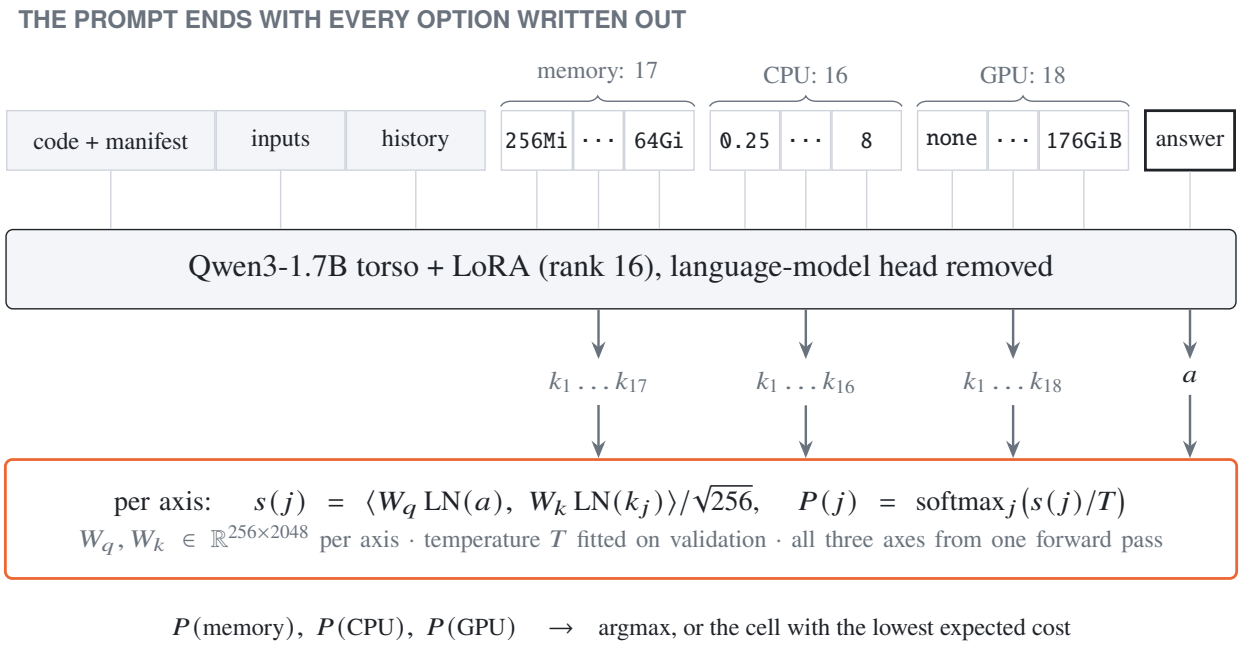


Figure 4: The pointer decision model. Every prompt ends with the same options block and answer instruction, so each option's last token sits at a fixed offset from the end, and one forward pass gives a distribution on all three axes.

negative log-likelihood. This is a pointer mechanism [23] over option tokens: the model reads the candidates in its context, so the same heads work for any option list written in the same format.

The torso is Qwen3-1.7B [17] without its language-model head, adapted with LoRA [10] (rank 16, α 32) on every projection of every layer. Prompts are cut from the left at 3,072 tokens, a length no evaluated prompt reaches. The loss is the sum of the three axes' cross-entropies, leaving out memory and CPU for GPU tasks. Training uses at most 5,000 rows per corpus and pod type, 27,936 rows in all, for one epoch with micro-batches of 8, two accumulation steps, and AdamW at 2×10^{-4} for the adapters and 10^{-3} for the heads, on one NVIDIA L4. Two ablations train on one pod type only, on half the rows for two epochs, so that all three variants take the same number of steps; each is calibrated on its own pod type's validation rows. The two pod types are drawn independently, so the model trained on both sees about 40% more distinct workloads than either ablation.

9.5 Turning a distribution into a request

The trees, the decision models and the pools all give a distribution on each axis, and we read each one out in two ways. The *argmax* read-out takes the most likely cell on each axis. The *decoded* read-out first decides whether to give a GPU, choosing none when $P(\text{none})$ is at least a threshold θ tuned on validation, and otherwise the most likely GPU cell. For memory it picks the cell with the lowest expected cost, $\arg\min_m \sum_j C[m, j]\, P(j)$, where $C[m, j]$ is RightSizeMem's memory cost of requesting cell m for a run that needs cell j, retries included (a fixed matrix, not the per-run simulator). For CPU it picks the cell most likely to fall within 25% of the busiest window. An OOM costs a retry, so decoding leans toward asking for more memory. For each model, validation decides which read-out the leaderboard reports (Table 22).

```
pi_ref = the supervised model's distributions, computed once before any update
for each batch of 8 prompts:
    one forward pass -> pi(.|x) on each axis
    sample K = 16 joint requests a^k = (m, c, g) from the three heads        # no text is generated
    r^k = - min( cost(x, a^k) / label_cost(x), 20 )                            # the benchmark's simulator
    A^k = (r^k - mean_k r) / (std_k r + 1e-3)                                  # group-normalized
    L   = - mean_k[ A^k * sum_{axis in S(x)} log pi(a^k_axis | x) ] + 0.05 * sum_{axis in S(x)} KL(pi ||
     pi_ref)
    S(x) = {gpu} for a GPU task, else all three axes
```

Figure 5: RL without generation. The reward divides by a fixed cost per run, so the best request for each run is the cheapest one whichever denominator is used. The KL term is exact over the categorical distributions.

9.6 Reinforcement learning against the cost model

Matching the label misses two things the cost model cares about. The axes interact: memory and CPU are free on a GPU node, while a GPU is the most expensive mistake on a CPU node. And asking for too little costs more than asking for too much. The RL approach starts from the decision model and maximizes the cost model's reward (Figure 5). Because the policy is three categorical distributions, a group of 16 joint requests comes from one forward pass and no text is generated. Advantages are normalized within each group as in GRPO [19], and each batch takes one on-policy step, so the update is REINFORCE with a group baseline and an exact KL penalty to the supervised model; there is no clipped ratio. It trains on 6,000 training prompts, about half of them unseen by the supervised model, for one epoch with a KL coefficient of 0.05 and the reward capped at 20.

The fitted temperatures of every RL model sit at or near the top of the search range (5.0), so its decoded read-out is set by that bound. The RL policy moves probability away from the label cell on purpose, so calibrating it against the label is the wrong target, and validation prefers its argmax read-out.

9.7 Opinion pools

The trees and the language models make different mistakes. The trees read numbers exactly and have nothing to go on for code they have never seen; the language models read code well and treat numbers as tokens. A logarithmic opinion pool [7] combines their calibrated distributions on each axis:

$$p_{\text{pool}}(j) \propto p_{\text{trees}}(j)^{w}\, p_{\text{LM}}(j)^{1-w}. \tag{2}$$

One weight w is shared across the three axes and tuned on validation together with θ, for the decoded read-out; nothing is retrained. A model trained on one pod type is pooled with trees trained on the same pod type.

9.8 A novelty gate

In our first release, the pooled RL model read by argmax was the best approach on familiar code and one of the worst on new code, while the RL model read out by decoding was the reverse. The gate sends each pod to one or the other by its distance from the training data:

$$s(x) = \tfrac{1}{10}\textstyle\sum_{i=1}^{10} \|z(x) - z(x_{(i)})\|_2, \qquad \text{request} = \begin{cases} \text{pool} \times \text{RL, argmax} & \text{if } s(x) \le \tau, \\ \text{RL, decoded} & \text{otherwise.} \end{cases} \tag{3}$$

Here z standardizes the trees' 209 features by their training mean and standard deviation, $x_{(i)}$ are the ten nearest training pods, and τ is the 95th percentile of s on validation. Validation has familiar families only, so it cannot tune the gate: the threshold sends 5% of familiar pods to the RL model by construction, and

validation regret falls steadily as fewer pods are routed. We chose the members, the signal and the threshold after seeing the first release's unseen results, and fixed them before training k8s-2 at new seeds. k8s-2 has the same seven held-out families, so it tests whether the choice holds up after relabelling and retraining, not whether it generalizes to new families.

Our first release divided a feature that never varied in training by 10^{-9}, so any pod with an unseen import landed effectively infinitely far away. Such features now keep their raw units; the old scaling remains as a comparison. We also compare two other signals: kNN on the 27 numeric features alone, and the Jensen–Shannon divergence between the trees' and the RL model's predictions.

9.9 A Flyte-to-Job adapter

A team whose history is all Flyte tasks has no Job pods to train on. The adapter shifts a Flyte-trained model's memory distribution with the accounting of Section 6.2: each cell's representative need n_j (its geometric middle) becomes $(n_j/1.05 - R + 48[\text{boto3}] + B[\text{tmpfs}]) \times 1.05$, using the manifest's volume medium, whether `job.py` imports boto3, and the input sizes. CPU and GPU come from the model's own reading of the Job's prompt. Its memory distribution can come from the model's reading of the Job's prompt (the variant a scheduler can deploy) or of the same run's Flyte task, which a scheduler holding only a raw Job does not have. We report both.

10 Experimental setup

The benchmark runs as one Flyte factory on an EKS cluster: a graph of cached tasks, each of which publishes a versioned artifact (Figure 6). The trees and rules fit on a CPU pod, each decision model and the RL model train on one NVIDIA L4, and the zero-shot model runs as a shared vLLM service in the same cluster. Each of these is itself a batch pod that someone had to size.

We trained k8s-2 three times on the same data (`rsk-k8s-2`, `-s2` and `-s3`). The seed controls data order, LoRA initialization and RL sampling; GPU kernels are not bit-for-bit deterministic, so even the same seed would not reproduce a model exactly. The zero-shot model is scored once. Tables show seed 1 with its bootstrap interval unless they say otherwise, and the leaderboard adds the mean and range over the three seeds. Seed 1 is the best or tied-best test seed for most trained approaches, so its numbers are slightly optimistic; Section 11.8 reports all three. Appendix D lists every cluster run.

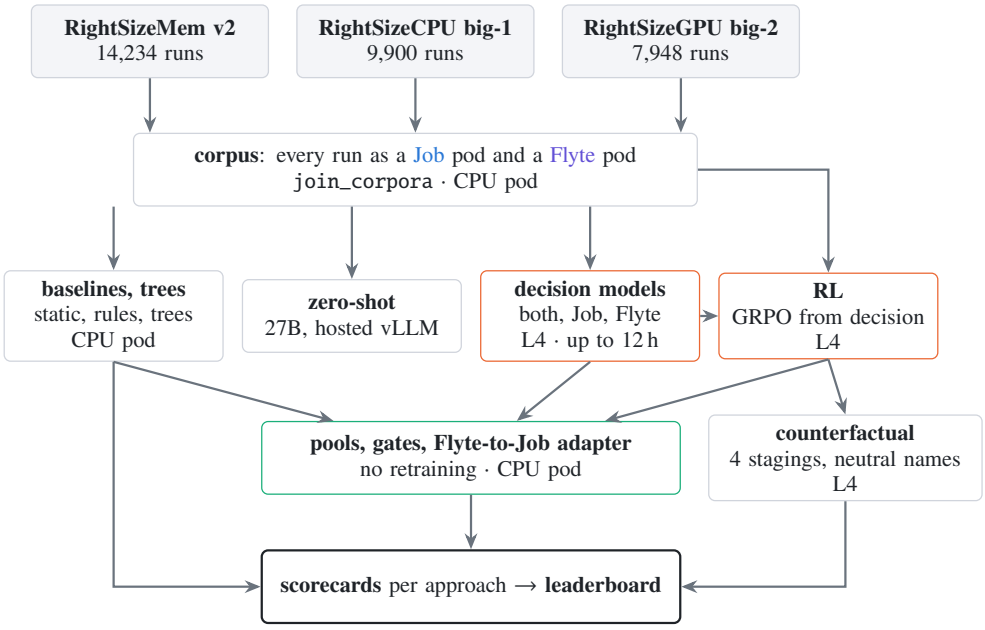


Figure 6: The benchmark as a Flyte factory. The three measured corpora are sources published by their own factories; every box below them is a cached task whose output is a versioned artifact partitioned by release (and by approach, for the per-approach boxes).

11 Results

We report Job pods first, because they are the plain Kubernetes case, with no orchestrator to absorb mistakes. Flyte pods follow, and so does the comparison between the two. Numbers are regret in percent against the cheapest request, seed 1, with 95% intervals clustered by task, unless noted.

11.1 Leaderboard

Table 9 and Figure 7 give the main results. On new tasks from familiar families, the trees pooled with the decision model are best at 11.7% [10.8, 12.7]. The trees pooled with the RL model and read by argmax (12.5%) cannot be separated from it (paired difference +0.8 points [-0.2, +1.9]). The gate (13.4%) and RL read by argmax (13.5%) are about two points behind. All four are ahead of the trees alone (16.8%) and the decision model alone (17.9%), and far ahead of the rules (44.5% for rules-sized, 49.2% for rules) and the zero-shot model (58.5%). A static request costs about twice the cheapest one.

On the seven held-out families every approach does worse, and the order changes. Rules-sized (42.7%) and the gate (42.8%) have the lowest regret, and the gate equals the decoded RL model it routes those pods to. The read-outs validation chose for the trained models do worse: RL read by argmax reaches 50.6%, the decision model 51.9%, the pools 61.0–84.1%, and the trees 73.8%. The zero-shot model is at 45.8% on its smaller sample. Clustered by family, the intervals on this split are wide: rules-sized spans [27.8, 60.4] and RL read by argmax [22.0, 83.5], and of the approaches in Table 9 only the static request is clearly worse than rules-sized. What does best on familiar code does poorly on new code, and on new code a hand-written rule that reads input sizes and the pod spec is as good as anything we trained.

The gate is the only trained approach near the top on both splits, at a cost of +0.9 points on familiar code [+0.6, +1.3] against its familiar-code member and a saving of -41.4 points on new code [-45.2, -37.6]. It

Table 9: Regret (%) on Job pods for seed 1, with 95% intervals clustered by task, and for the unseen split also by family; regret on Flyte pods; dollar regret on Job pods; and the mean and range of Job regret over three trainings. Each trained model is shown with the read-out validation chose (Table 22). Bold marks the approaches whose paired difference from the column's best includes zero. Each split has 5,095 Job pods for test and 6,416 for unseen. †Zero-shot is scored on a sample of 130 test and 150 unseen Job pods (Table 24) and is not compared for bold.

	Job pods: regret (%)			Flyte pods		Job $ regret		Job, 3 seeds: mean (range)	
approach	test	unseen	by family	test	unseen	test	unseen	test	unseen
static	97.8 [93.4, 102.1]	120.7 [116.6, 124.6]	[76.2, 167.6]	100.0	118.5	188.7	260.7	97.8	120.7
rules	49.2 [44.8, 53.6]	53.3 [50.5, 56.1]	[27.8, 84.0]	47.7	51.8	60.0	121.8	49.2	53.3
rules-sized	44.5 [40.4, 49.1]	**42.7** [40.6, 44.7]	[27.8, 60.4]	43.7	**44.9**	59.0	119.4	44.5	42.7
zero-shot 27B†	58.5 [41.2, 77.5]	45.8 [31.9, 60.7]	[18.5, 79.7]	43.2	56.4	89.6	30.5	—	—
zero-shot 27B, JSON†	58.0 [40.3, 77.0]	47.0 [33.6, 62.8]	[14.4, 82.4]	27.7	54.4	104.7	82.6	—	—
trees	16.8 [15.5, 18.1]	73.8 [70.9, 76.7]	[30.4, 132.5]	19.8	73.9	33.2	86.9	16.8	73.8
decision	17.9 [16.5, 19.2]	51.9 [49.6, 54.2]	[24.7, 85.3]	16.4	47.2	42.4	120.5	18.9 (17.9–20.3)	53.6 (48.3–60.7)
RL, argmax	13.5 [12.1, 14.9]	50.6 [48.3, 53.0]	[22.0, 83.5]	13.6	50.6	38.0	107.9	13.4 (13.1–13.6)	49.0 (45.0–51.4)
pool × decision	**11.7** [10.8, 12.7]	61.0 [58.5, 63.4]	[26.5, 106.6]	**10.5**	55.7	30.2	96.4	11.9 (11.7–12.0)	59.9 (56.3–62.5)
pool × RL, argmax	**12.5** [11.3, 13.8]	84.1 [80.4, 87.9]	[31.3, 150.4]	12.2	83.1	18.1	113.1	14.0 (12.1–17.5)	94.6 (83.5–116.0)
gate	13.4 [12.2, 14.7]	**42.8** [40.7, 44.8]	[24.2, 65.1]	12.7	**44.0**	21.9	129.5	15.0 (13.3–18.4)	41.1 (38.7–42.8)

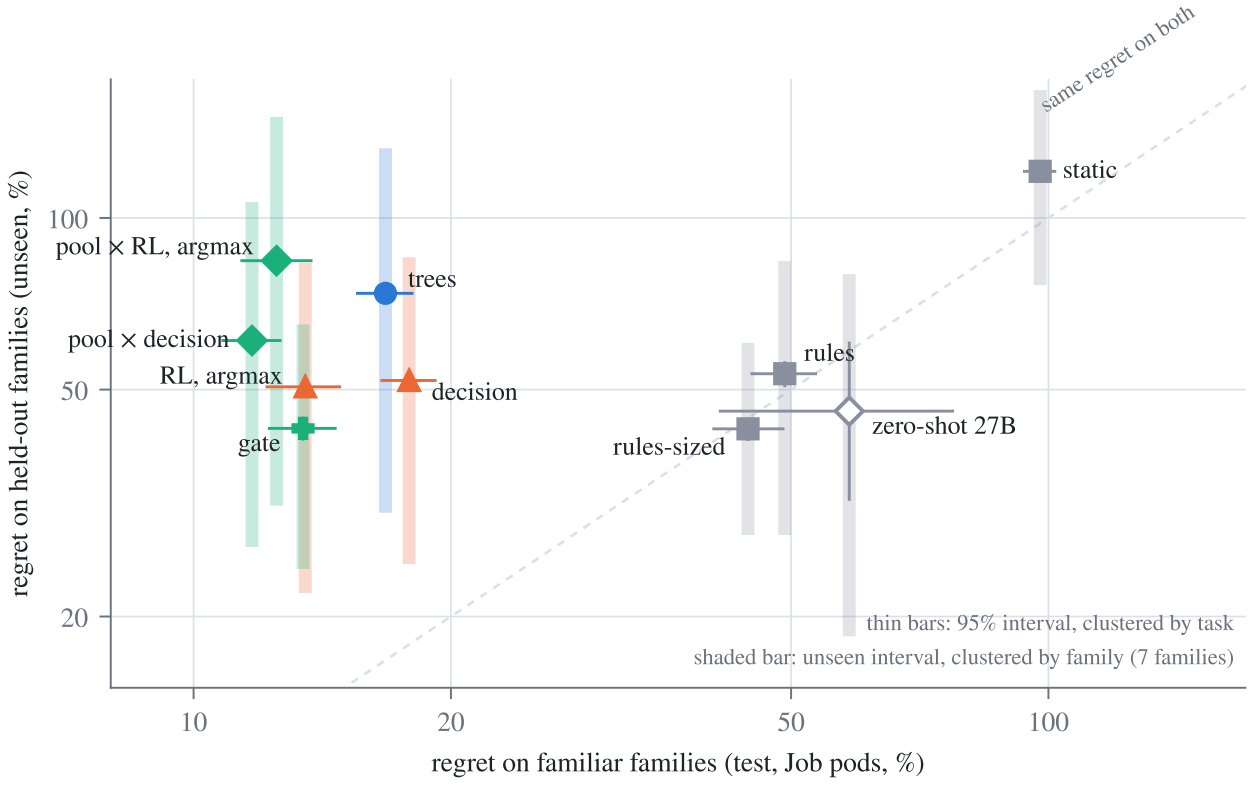


Figure 7: Regret on familiar families against regret on held-out families, Job pods, seed 1 (log scales). Gray: static request, rules and zero-shot (hollow: scored on a sample). Blue: trees. Orange: language models. Green: pools and the gate. No approach is in the lower left: the approaches that lead on familiar code fall behind a sized rule on new code.

Table 10: Job regret (%) on test pods whose task code appears in training (seen) and on those whose code does not (new), with the unseen families for comparison, for seed 1.

	test: code in training?		
approach	seen (4,135)	new (960)	unseen families
static	86.1	148.0	120.7
rules	52.0	36.7	53.3
rules-sized	44.5	44.4	42.7
trees	15.7	21.4	73.8
decision	14.0	34.3	51.9
RL, argmax	11.6	21.8	50.6
pool × decision	10.3	17.9	61.0
pool × RL, argmax	12.4	13.2	84.1
gate	12.6	16.8	42.8

does not beat rules-sized on new code (-0.1 points [-1.6, +1.3]), and its members were chosen with the same held-out families in view (Section 11.5).

11.2 Seen code, new programs and new families

Most familiar-family test tasks are programs the models have seen. 81% of test Job pods run code identical to a training task's, and differ only in input sizes and history. Table 10 splits the test split in two. On the 960 pods with new programs, regret rises for every trained approach. The decision model goes from 14.0% to 34.3%. The trees pooled with the RL model and read by argmax barely move (12.4% to 13.2%), and the gate goes from 12.6% to 16.8%. The held-out families are the larger step: new programs from familiar families stay within 13–35% of the cheapest request, while every approach is at about 43% or worse on the new families. Rules-sized reads only sizes and the pod spec, so it is the same on both halves.

Part of the gap between test and unseen comes from the corpus mix: GPU runs are 36% of unseen pods and 22% of test pods. Table 23 in Appendix E breaks results down by corpus. The trees' weakness on new code is concentrated in CPU and GPU families (87.3% and 103.3%), and rules-sized is strongest on the held-out CPU families (24.5%).

11.3 Does a model need to see Jobs?

Table 11 trains the trees, the decision model and the pool three ways: on both pod types, on Jobs alone, and on Flyte tasks alone. Flyte data does not make the models better on Jobs. A decision model trained only on Jobs scores 18.5% on Job test pods, against 17.9% for one trained on both, a difference of -0.6 points [-1.9, +0.7]; on seed 3 the Job-only model is better. The trees and the pool behave the same way. The reverse transfer does not work. A model trained only on Flyte tasks scores 43.6% on Job pods, and the pool built on it reaches 30.6% where the pool trained on both reaches 11.7%. A model trained only on Jobs also does worse on Flyte pods (25.9% against 16.4%), by less, because a Flyte task is closer to a Job with a small fixed runtime added than the other way around. On the unseen families, the model trained on both pod types is ahead by -8.4 points [-10.5, -6.3], which may reflect the larger number of distinct workloads it saw.

A model trained on both pod types treats them almost the same. The paired difference between a run's Job pod and its Flyte pod is under two points on familiar code for every language model and pool (Table 18), and for the RL model it is indistinguishable from zero; the trees do 3.0 points better on Jobs. On new code, the decision model and its pool do about five points worse on Jobs than on Flyte tasks, while RL and the gate do about a point better.

Table 11: What the training data includes. Regret (%) on Job and Flyte pods, and the share of Job pods whose first attempt succeeds, for seed 1.

		Job pods: regret (%)		Flyte pods: regret (%)		Job
approach	trained on	test	unseen	test	unseen	first OK
trees	both	16.8 [15.5, 18.1]	73.8	19.8 [18.5, 21.1]	73.9	97.6
	Jobs only	17.2 [16.0, 18.4]	73.4	33.0 [31.0, 35.0]	84.6	97.7
	Flyte only	24.0 [22.4, 25.8]	77.7	21.2 [19.8, 22.6]	98.6	94.1
decision	both	17.9 [16.5, 19.2]	51.9	16.4 [15.0, 17.7]	47.2	97.2
	Jobs only	18.5 [17.1, 19.9]	60.3	25.9 [24.3, 27.6]	57.7	97.0
	Flyte only	43.6 [41.3, 46.0]	63.2	17.1 [15.7, 18.5]	53.1	87.2
decision, argmax	both	36.7 [34.4, 39.1]	74.1	34.9 [32.7, 37.3]	68.8	87.3
	Jobs only	35.4 [33.3, 37.6]	79.5	48.0 [45.3, 50.6]	84.6	87.0
	Flyte only	63.8 [60.8, 66.8]	89.7	33.6 [31.6, 35.7]	72.2	76.9
pool × decision	both	11.7 [10.8, 12.7]	61.0	10.5 [9.6, 11.5]	55.7	98.3
	Jobs only	12.8 [11.7, 13.9]	62.3	22.8 [21.3, 24.4]	63.2	98.0
	Flyte only	30.6 [28.8, 32.7]	55.6	10.4 [9.5, 11.4]	54.2	91.3

The adapter. The adapter of Section 9 shifts a Flyte-trained model's memory distribution through the staging accounting. The variant that reads the run's Flyte task cuts the Flyte-only model's Job regret from 43.6% to 34.6% on seed 1, from 40.1% to 28.1% on seed 2, and from 44.2% to 34.9% on seed 3. The variant a scheduler can deploy, which reads only the Job's prompt, does not help (44.9%, 40.2% and 43.4%), though pooling it with Flyte-trained trees does (29.6%, 26.1% and 26.9%). Either way the adapter stays well short of a model that has seen Jobs (17–20%): platform data matters even when the accounting is known.

11.4 Does a model read the manifest?

A model could do well on Jobs by learning only that a Job's numbers run lower than a Flyte task's, because a Job does not carry the runtime. We test this two ways.

Run and Job pairs. Of the 3,950 test pairs of a run's Flyte pod and its Job pod on CPU nodes, the memory label differs in 29.9%. In 638 pairs the Job needs more, because it stages its inputs on tmpfs, and in 543 it needs less, because it has no runtime. A model that ignores the pod type keeps the same request for both pods, which scores 100% on the pairs whose label stays the same and 0% on those whose label moves. Models trained on both pod types move the right way on 56–67% of the pairs that move, and on 75–83% of upward moves (Table 19). The rules move the right way on 25%.

A controlled counterfactual. The pairs above mix the effect of the manifest with the effect of the runtime. To isolate the manifest, we took 1,189 first runs from the test and unseen splits and rendered each one as a Job under all four stagings, holding the task, code and inputs fixed (4,756 pods). Only the manifest and the way the job fetches its inputs change, and with no history, the manifest is the only evidence of the staging. Switching the volume from disk to tmpfs moves the label in 48.6% of test pairs and 74.2% of unseen pairs, and switching the fetcher from an init container to boto3 moves it in about a fifth.

The models trained on Jobs read the manifest (Figure 8 and Table 25). When the label moves, the decision model, the trees, RL and the pools move their request the right way on 72–78% of familiar-code pairs and 64–78% of unseen pairs. The Flyte-only model, which has never seen a manifest, does so on 21%. Static requests and rules never move, because on a first run they have only fixed defaults. On new code the models

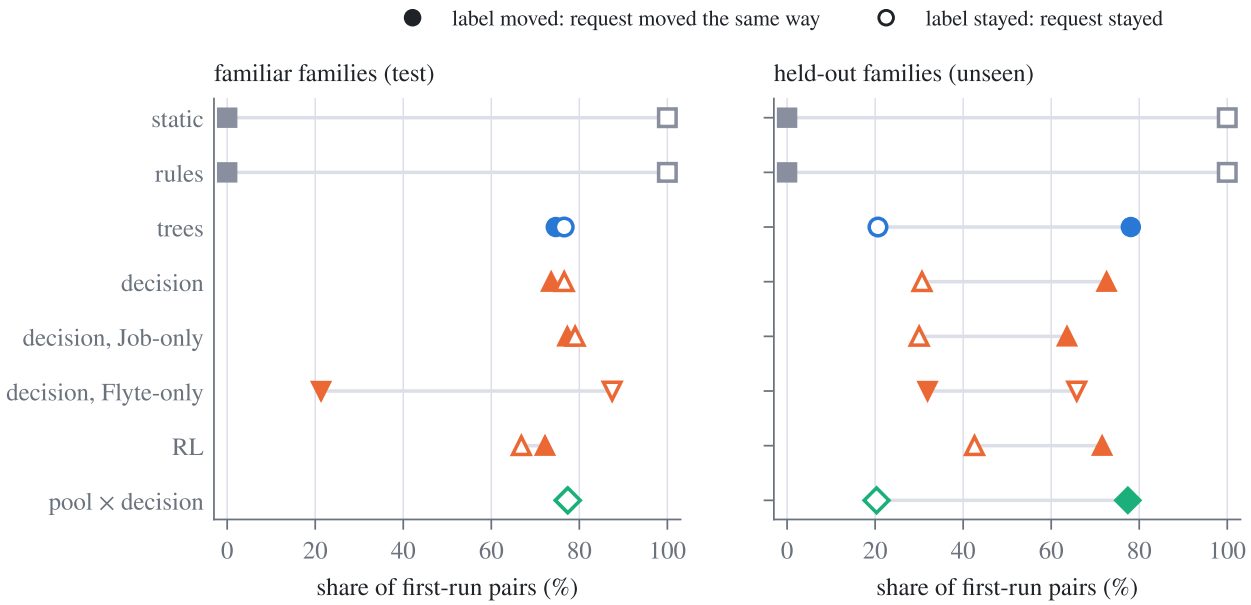


Figure 8: The staging counterfactual, seed 1: the same first runs rendered as Jobs with the inputs on node disk and on a memory-backed volume. Filled: among pairs whose label moved, the share where the request moved the same way. Hollow: among pairs whose label stayed, the share where the request stayed too. Static requests and rules never move on a first run.

overreact: when the label does not move, which mostly means the staged inputs were small, they still change their request on 57–80% of unseen pairs. They pick up the fetcher less reliably (31–47% on test), which fits boto3's 48 MiB only sometimes crossing a cell.

The task's name. Generated tasks are named after their family (for example `pandas-join`), and the name appears in the code and the manifest. We re-scored the counterfactual's first runs with each name replaced by a hash, without retraining (Table 20). On familiar code, regret changes by less than a point for every trained approach except the Flyte-only model (+1.8); on new code it rises by 0.5–3.6 points. Each family also has a fixed function name and input schema, which this test leaves in place, so it bounds the name's effect without removing the family's identity from the code.

11.5 The gate

Table 12 compares the gate's signal with the alternatives. Correcting the first release's kNN scaling changed nothing: the corrected distance still separates unseen families from validation with an AUROC of 0.999, sends every unseen pod to RL, and gives exactly the same requests. The held-out families are far from the training data in code tokens, though only one of them imports a library no training family uses (Section 5). With code tokens removed the AUROC falls to 0.67 and unseen regret rises to 66.4%. The detector's job is easy on this benchmark, and the benchmark cannot test it on new code built from familiar tokens.

Over three trainings, the gate averages 15.0% on familiar code (13.3–18.4%) and 41.1% on new code (38.7–42.8%). Seed 3's familiar-code number is higher because its familiar-code member, the pooled RL model read by argmax, was worse on that seed (17.5%). On new code the gate always equals the decoded RL model, whose temperature sits at its search bound (Section 9); validation would have chosen that model's argmax read-out, which is about eight points worse on new code (+7.8 points [+6.2, +9.5]).

Table 12: Gate signals for seed 1. AUROC measures how well each signal ranks unseen pods above validation pods. The middle columns give the share of pods routed to the RL model, and the last two the gate's regret on Job pods. All gates use the 0.95 quantile except the disagreement gate, shown at 0.5.

		sent to RL (%)		Job regret (%)	
signal	AUROC	test	unseen	test	unseen
kNN, all features	0.999	5.3	100.0	13.4 [12.3, 14.6]	42.8 [40.8, 44.8]
kNN, k8s-1 scaling	1.000	5.3	100.0	13.4 [12.2, 14.7]	42.8 [40.6, 44.8]
kNN, numeric only	0.673	4.5	30.5	13.2 [12.0, 14.5]	66.4 [63.5, 69.4]
disagreement (JS), q 0.5	0.729	51.3	80.5	16.6 [15.3, 18.1]	46.6 [44.4, 48.8]

Table 13: Job regret (%) on first runs and on runs with history, for seed 1.

	test		unseen	
approach	first run	with history	first run	with history
static	107.7	90.4	137.4	106.2
rules	74.7	30.0	77.0	32.8
rules-sized	81.5	16.8	72.4	**16.9**
zero-shot 27B	98.0	24.6	59.6	36.1
zero-shot 27B, JSON	88.5	31.8	81.2	22.9
trees	22.0	12.9	115.5	37.6
decision	27.5	10.7	81.7	26.1
RL, argmax	23.2	**6.2**	80.5	24.6
pool × decision	**16.4**	8.2	103.9	23.8
pool × RL, argmax	19.3	7.4	145.9	30.6
gate	20.3	8.3	**65.2**	23.3

11.6 First runs and runs with history

First runs are where the approaches differ most (Table 13). With history, most trained approaches do well on familiar code (6–13%), and rules-sized reaches 16.8%. On first runs the rules lose most of their accuracy (72–82%), while the pooled decision model holds at 16.4%. On new code, history does most of the work, and rules-sized with history (16.9%) is the best of any approach, because scaling the nearest earlier run by input size is a strong prior when a task's runs differ only in size. Without history, no approach does well on new code; the best are the zero-shot model, on its sample, and the gate.

11.7 Dollars and the price of a retry

Mean regret counts every pod equally, while dollar regret is dominated by large jobs. On familiar code the two mostly agree: the pooled RL model read by argmax costs 18.1% more than the cheapest requests in total, and the gate 21.9%. On new code, dollars rank the trees first (86.9%), because they size the largest jobs better even though their mean regret is among the worst. The trees cost $959 per 1,000 unseen Job runs, against $1,126 for rules-sized, $1,177 for the gate, and $513 for the cheapest requests.

A raw Job with `backoffLimit: 0` does not retry: it fails, and someone resubmits it. That makes an OOM much more expensive than our default penalty. Figure 9 and Table 14 re-score every approach with penalties of 0, 5 and 20 times the label request's cost per retry. Every approach was tuned at the default penalty of 1, so this measures how fixed requests hold up when failures get dearer, not how well each approach could adapt. On familiar code the pooled decision model has the lowest regret at every penalty from 1 to 20 (95.3% at 20), though at 20 the pooled RL model is within -13.4 points [-27.2, +0.7]. On new code,

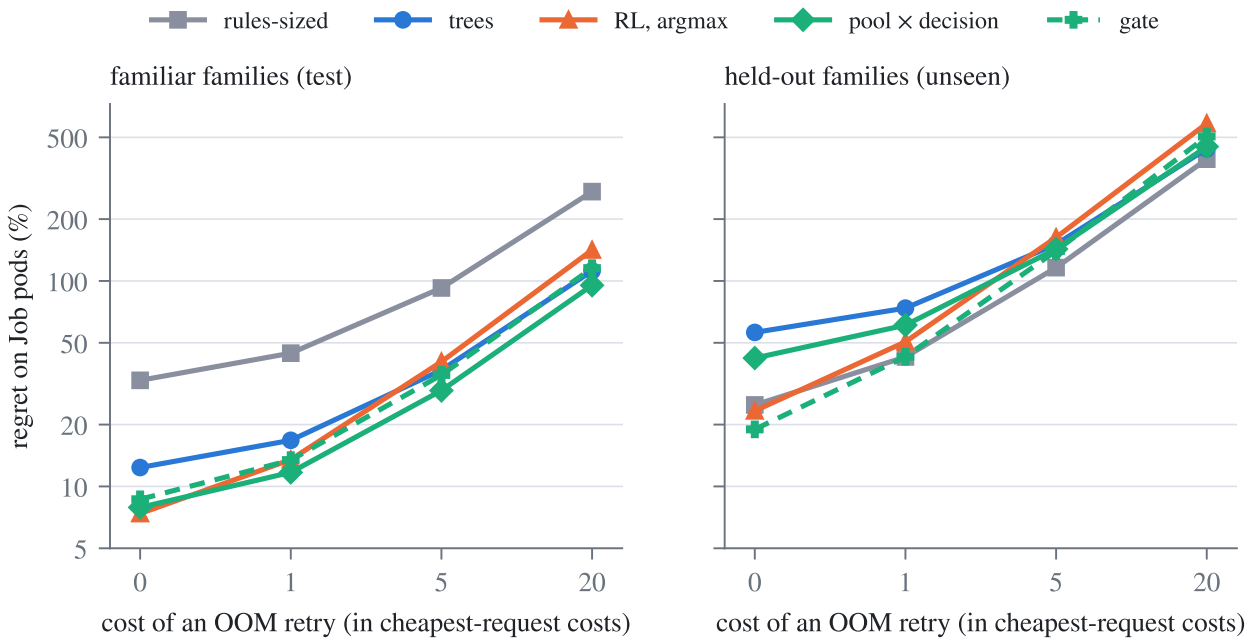


Figure 9: Job regret as an OOM retry gets more expensive, seed 1 (log scale). Every approach was tuned at a penalty of 1. On familiar code the trained approaches stay far ahead of rules-sized; on held-out families the lines cross, and rules-sized has the lowest regret once a retry costs 5 or more.

Table 14: Job regret (%) when each OOM retry costs 0, 1 (the default), 5 or 20 times the label request's cost. Higher penalties stand for a Job that fails outright and has to be resubmitted.

	0		1 (default)		5		20	
approach	test	unseen	test	unseen	test	unseen	test	unseen
static	56.3	50.2	97.8	120.7	264.2	404.1	888.2	1466.6
rules	35.7	28.8	49.2	53.3	104.3	153.9	311.2	531.1
rules-sized	32.8	24.9	44.5	**42.7**	92.4	**116.1**	272.1	**391.4**
zero-shot 27B	44.2	24.3	58.5	45.8	131.2	143.9	403.9	511.5
zero-shot 27B, JSON	39.1	29.8	58.0	47.0	149.8	127.8	493.9	430.9
trees	12.4	56.2	16.8	73.8	36.8	150.8	112.0	439.7
decision	13.1	24.6	17.9	51.9	39.5	165.1	120.6	589.9
RL, argmax	**7.4**	23.3	13.5	50.6	40.4	162.8	141.4	583.9
pool × decision	7.9	42.1	**11.7**	61.0	**29.3**	143.0	**95.3**	450.8
pool × RL, argmax	8.0	59.5	12.5	84.1	32.7	188.7	108.7	580.8
gate	8.7	**19.0**	13.4	42.8	34.9	140.1	115.4	505.1

rules-sized has the lowest regret at penalties of 5 and 20 (116.1% and 391.4%), ahead of RL and the gate (140.1% and 505.1%). Clustered by task, that lead is clear (-113.7 points [-129.8, -97.4] at 20); clustered by the seven families, it is not ([-263.7, +15.9]). For teams that run raw Jobs without retries, the evidence favours a rule that scales by input size and reads the manifest on code nobody has seen before, and does not rule out the trained models.

Table 15: Job regret (%) for each of the three trainings. The static request, the rules and the trees give identical results on every seed.

	Job test, by seed			Job unseen, by seed		
approach	1	2	3	1	2	3
static	97.8	97.8	97.8	120.7	120.7	120.7
rules	49.2	49.2	49.2	53.3	53.3	53.3
rules-sized	44.5	44.5	44.5	42.7	42.7	42.7
trees	16.8	16.8	16.8	73.8	73.8	73.8
decision	17.9	18.6	20.3	51.9	48.3	60.7
decision, argmax	36.7	36.8	38.2	74.1	73.1	85.8
RL	19.6	21.8	22.6	42.8	41.7	38.7
RL, argmax	13.5	13.6	13.1	50.6	51.4	45.0
pool × decision	11.7	11.9	12.0	61.0	56.3	62.5
pool × decision, argmax	20.3	20.5	20.7	89.2	85.4	101.7
pool × RL	15.5	15.5	16.0	53.5	54.9	65.6
pool × RL, argmax	12.5	12.1	17.5	84.1	83.5	116.0
gate	13.4	13.3	18.4	42.8	41.7	38.7
decision, Job-only	18.5	19.2	17.5	60.3	57.6	73.2
decision, Flyte-only	43.6	40.1	44.2	63.2	71.8	81.3
trees, Job-only	17.2	17.2	17.2	73.4	73.4	73.4
trees, Flyte-only	24.0	24.0	24.0	77.7	77.7	77.7

11.8 Three trainings

The static request, both rules and the trees give identical results on every seed. The trained models vary, more on new code than on familiar code (Table 15). On familiar code the top is stable: the pooled decision model ranges from 11.7% to 12.0% and RL read by argmax from 13.1% to 13.6%. The pooled RL model read by argmax ranges from 12.1% to 17.5%. On new code, the decision model ranges from 48.3% to 60.7% and the pooled RL model read by argmax from 83.5% to 116.0%, while the gate varies least (38.7–42.8%). Rankings on new code within about ten points of each other are not stable across trainings.

11.9 Zero-shot

With room to finish its reasoning, the 27B model answers almost every prompt. Table 24 scores every approach on the zero-shot model's own sample, so the comparison is like for like. On familiar code the zero-shot model (58.5%) is worse than the rules (39.1% and 34.0%; paired difference with rules-sized +24.4 points [+5.9, +45.7]) and far behind the trained models (12.0% for RL read by argmax). On new code it is level with the best approaches on that sample (45.8% against 40.4% for rules-sized and 43.5% for the gate), with intervals of about ±15 points. Constraining it to the JSON schema makes little difference on Jobs and helps on Flyte tasks (27.7% on familiar code).

12 Release

The benchmark and the models are on the Hugging Face Hub, in the `unionai` organization's "RightSize Bench" collection, and the code that builds and scores them is at `https://github.com/unionai-oss/model-factory`.

The dataset (unionai/rightsize-bench, CC-BY-4.0) has one row per pod, 64,164 in all, split into train, validation, test and unseen. Each row holds what an approach may read (the task code, the Job manifest, the input metadata, the history, and the rendered prompt), the labels as grid cells and as option text, the cheapest request and its cost, and the columns the cost simulator replays. Configurations give the Job and Flyte pods separately, the 209 tree features, the counterfactual renditions, and every reported approach's request and cost for each pod and seed, so a new approach can be compared with ours pod by pod. The simulator is resource_tuner/bench_all/scoring.py; it runs on the published rows unchanged.

The models (Apache-2.0) are the trees, in scikit-learn's skops format; the decision model and the RL model; and the two training-data ablations:

- unionai/rightsize-trees
- unionai/rightsize-decision-qwen3-1.7b and unionai/rightsize-rl-qwen3-1.7b
- unionai/rightsize-decision-qwen3-1.7b-job-only and -flyte-only

Each decision-model repository holds the LoRA adapter, the pointer heads in safetensors, the calibration, and a standalone inference file that needs only transformers and peft; seeds 2 and 3 are branches. A unit test checks that the inference file reproduces the training code's scores exactly in fp32.

Intended use. The benchmark is for comparing ways of choosing a pod's resource request before it runs. It is not a model of any production cluster: the workloads are generated and short, prices are list prices, and the labels have the biases of Section 7. The models are baselines for that comparison and have not been tested on production workloads.

Contamination. The test and unseen labels are public. A model trained on them, or on this paper's predictions, is no longer evaluated on new tasks; this release has no private held-out set.

Maintenance. Every release is a tagged revision of each repository (this one is k8s-2), and its factory run is recorded in the project's research log. The earlier releases (RightSizeMem v2, RightSizeCPU big-1, RightSizeGPU big-2, and k8s-1) are described in Appendix D.

Environment. Runs were measured on Amazon EKS under cgroup v2, on spot c5.4xlarge nodes for the CPU corpora and on the GPU shapes of Table 2, with pandas 3.0.5, polars 2.0.0, DuckDB 1.5.6, PyTorch 2.14.0 and scikit-learn 1.9.0. The inputs are synthetic and contain no personal data.

13 Discussion

One measurement, two kinds of pod. The user code is the same whichever way a pod is submitted, so one measurement of the workload can label both a raw Job and a Flyte task, as long as the accounting for the pod's edges is right. Getting it right took real pods. Sibling processes in one pod confirmed the arithmetic. Only per-container cgroups showed where tmpfs pages are charged, that the first release's init containers could not hold their inputs, and that the Flyte runtime was less than half the size we had assumed. Labels derived this way should be checked against the cgroup, since a second RSS measurement repeats the first one's biases.

What a model learns from Jobs. A model trained on Jobs reads the manifest. It changes its memory request when the volume medium changes, on first runs where nothing else gives the staging away. Flyte tasks add little to that model's accuracy on Jobs, and a Flyte-only model transfers poorly; an adapter that applies the accounting helps only when it can also see the run's Flyte task. For a team that submits pods both ways, the practical advice is to train on the pods it actually runs.

Familiar code, new programs and new families. The benchmark separates three levels of novelty, and the models handle them differently. Most familiar-family test tasks repeat a training program at a new input size, which every trained approach handles well. New programs from familiar families cost the decision model the most and the pooled RL model almost nothing. New families cost every approach, and there a rule that reads input sizes and the pod spec is as good as anything we trained. Pooling the trees with a language model helps on familiar code and hurts on new families, because the pool's weight is tuned on familiar families. A gate can recover the better approach on each split when its detector works, and on this benchmark the detector's job is easy. A benchmark with held-out families built only from familiar code tokens would test it properly.

Cost, not accuracy. The read-out that matches labels best is not the one that costs least. Cost-aware decoding over-asks memory on purpose. The label itself is not the cheapest request: one CPU cell below the busiest window is often cheaper, which is why our first release, scored against the label, reported negative regret. And the ranking on new code flips when retries get expensive. A benchmark for this decision has to score money, and it has to say what a failure costs.

14 Limitations

The tasks. The workloads are synthetic. They use real libraries and real code paths, but the tasks are generated, the data is random, and runs take seconds to minutes. Several families have only a handful of distinct programs, so most of the familiar-family test split repeats training code. The seven held-out families were chosen by hand, two of them are DuckDB families from different corpora, and only DuckDB is a library the training families never import. The code names the family through task names, function names and input schemas; the neutral-name test removes only the first. The people who wrote the generators also wrote the tree features and the rules.

The labels. The `ru_maxrss` floor and memory-mapped inputs make about a third of memory labels one cell larger than needed; on tmpfs stagings a mapped input may be counted twice. Process pools are labelled by their largest process, which is why two of 100 real pods failed at their label. RightSizeMem has no per-second CPU series, so its throttling is modelled from mean cores. The Job labels and the Job history are derived from the workload measurement with the same accounting, and were checked on 100 runs in real pods and 176 in one pod; no Flyte pod was run with its limit at its label, and GPU runs were not re-measured as Jobs. Most multi-GPU runs were censored because the cluster had no capacity for them, single-GPU runs that outgrew a T4 were measured on an L40S, and fits on unmeasured machines are modelled, so the GPU labels above a T4 rest partly on the fit model.

The evaluation. The cost model uses list prices with no spot discounts or bin-packing, and every approach was tuned at a retry penalty of 1; we re-priced fixed requests at other penalties without re-tuning. Generalization to new kinds of code rests on seven families, so family-clustered intervals on the unseen split are wide, and the gate's members were chosen with those families in view. The RL model's calibration temperature sits at its search bound. We trained three times, enough to show that rankings on new code within about ten points are not stable and not enough to bound that variation. The zero-shot model was scored once, on 130 test and 150 unseen Job pods.

15 Conclusion

RightSize Bench scores the request a scheduler makes before a Kubernetes pod runs, in dollars against the cheapest request, across 33 families of task code submitted both as raw Jobs and through Flyte. Checking its labels in real pods fixed two errors and measured Flyte's runtime at about 70 MiB. Gradient-boosted trees pooled with a small decision model trained on Jobs come within 12% of the cheapest request on familiar code, and the decision model reads the manifest. On held-out families a careful rule does as well as any model we trained, and it holds up best when failures are expensive. The open problems are new programs and new families, and a benchmark whose held-out code does not announce itself through new tokens. The dataset, the models and the factory that builds them are public.

References

[1] Eli Cortez, Anand Bonde, Alexandre Muzio, Mark Russinovich, Marcus Fontoura, and Ricardo Bianchini. Resource central: Understanding and predicting workloads for improved resource management in large cloud platforms. In *Proceedings of the 26th Symposium on Operating Systems Principles (SOSP)*, 2017. doi: 10.1145/3132747.3132772.

[2] Christina Delimitrou and Christos Kozyrakis. Quasar: Resource-efficient and QoS-aware cluster management. In *Proceedings of the 19th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2014. doi: 10.1145/2541940.2541941.

[3] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall, 1993.

[4] Christopher A. Field and Alan H. Welsh. Bootstrapping clustered data. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 69(3):369–390, 2007. doi: 10.1111/j.1467-9868.2007.00593.x.

[5] Flyte Authors. Flyte. `https://flyte.org`, 2025.

[6] Jerome H. Friedman. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5):1189–1232, 2001. doi: 10.1214/aos/1013203451.

[7] Christian Genest and James V. Zidek. Combining probability distributions: A critique and an annotated bibliography. *Statistical Science*, 1(1):114–135, 1986. doi: 10.1214/ss/1177013825.

[8] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017.

[9] Tejun Heo. Control group v2. Linux kernel documentation, `https://docs.kernel.org/admin-guide/cgroup-v2.html`, 2024.

[10] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

[11] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[12] Roger Koenker and Gilbert Bassett. Regression quantiles. *Econometrica*, 46(1):33–50, 1978. doi: 10.2307/1913643.

[13] Kubernetes Authors. Resource management for pods and containers. https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/, 2024.

[14] Kubernetes Authors. Volumes: emptyDir. https://kubernetes.io/docs/concepts/storage/volumes/#emptydir, 2024.

[15] Kubernetes Authors. Vertical pod autoscaler. https://github.com/kubernetes/autoscaler/tree/master/vertical-pod-autoscaler, 2024.

[16] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023. doi: 10.1145/3600006.3613165.

[17] Qwen Team. Qwen3 technical report, 2025.

[18] Krzysztof Rzadca, Pawel Findeisen, Jacek Swiderski, Przemyslaw Zych, Przemyslaw Broniek, Jarek Kusmierek, Pawel Nowak, Beata Strack, Piotr Witusowski, Steven Hand, and John Wilkes. Autopilot: Workload autoscaling at Google. In *Proceedings of the Fifteenth European Conference on Computer Systems (EuroSys)*, 2020. doi: 10.1145/3342195.3387524.

[19] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models, 2024.

[20] Yiyou Sun, Yifei Ming, Xiaojin Zhu, and Yixuan Li. Out-of-distribution detection with deep nearest neighbors. In *Proceedings of the 39th International Conference on Machine Learning (ICML)*, 2022.

[21] Muhammad Tirmazi, Adam Barker, Nan Deng, Md E. Haque, Zhijing Gene Qin, Steven Hand, Mor Harchol-Balter, and John Wilkes. Borg: The next generation. In *Proceedings of the Fifteenth European Conference on Computer Systems (EuroSys)*, 2020. doi: 10.1145/3342195.3387517.

[22] Abhishek Verma, Luis Pedrosa, Madhukar Korupolu, David Oppenheimer, Eric Tune, and John Wilkes. Large-scale cluster management at Google with Borg. In *Proceedings of the Tenth European Conference on Computer Systems (EuroSys)*, 2015. doi: 10.1145/2741948.2741964.

[23] Oriol Vinyals, Meire Fortunato, and Navdeep Jaitly. Pointer networks. In *Advances in Neural Information Processing Systems*, volume 28, 2015.

A Task families

Table 16 lists every family with its resource driver and one sample input, rendered as an approach sees it (seed 7). "Unseen" families are held out of training and validation. "Sharded" families spread over every visible GPU. Lines is the length of the sample's function body.

Table 16: The 33 task families.

corpus	family	driver	sample input	lines
RightSizeMem	csv_chunked	code	readings: csv, header: device,x0,x1,x2,x3,x4,x5,x6,x7,tag0	13
	jsonl_records	both	logs: jsonl, first record keys: ['user', 'score0', 'scor...	17
	numpy_linalg	both	matrix: npy, shape=(326345, 128), dtype=float64	15
	pandas_groupby	both	events: parquet, 1,790,583 rows, schema=[customer_id: i...	14
	pandas_join	both	orders: parquet, 632,344 rows, schema=[order_id: int64,...	19
	polars_pipeline	code	events: parquet, 2,236,012 rows, schema=[user_id: int64...	15
	python_grouping	both	rows: csv, header: key,value,note0,note1,note2	18
	sklearn_fit	both	train: parquet, 326,345 rows, schema=[x0: double, x1: do...	19
	tfidf_text	both	docs: parquet, 52,765 rows, schema=[doc_id: int64, text...	16
	torch_train	both	train: parquet, 1,710,807 rows, schema=[x0: double, x1:...	39
	duckdb_sql (unseen)	code	events: parquet, 2,403,570 rows, schema=[account_id: in...	15
	image_batch (unseen)	both	images: npy, shape=(52998, 128, 128, 3), dtype=uint8	15
RightSizeCPU	io_bound	io	events: jsonl, first record keys: ['user', 'event', 'pay...	27
	mixed_phases	phases	matrix: npy, shape=(101438, 256), dtype=float64	28
	numpy_blas	threads	features: npy, shape=(135042, 512), dtype=float32	25
	pandas_serial	serial	orders: parquet, 893,225 rows, schema=[customer_id: int...	20
	polars_groupby	threads	events: parquet, 5,464,904 rows, schema=[key: int64, reg...	22
	process_pool	processes	docs: csv, header: id,text	31
	python_threadpool	gil	blob: npy, shape=(1773, 65536), dtype=uint8	29
	sklearn_ensemble	threads	train: parquet, 177,376 rows, schema=[f0: double, f1: do...	22
	torch_cpu	threads	train: parquet, 177,376 rows, schema=[f0: double, f1: do...	32
	duckdb_sql (unseen)	threads	ledger: parquet, 5,464,904 rows, schema=[account_id: in...	22
	pyarrow_csv (unseen)	threads	readings: csv, header: sensor,r0,r1,r2,r3	24
RightSizeGPU	blocked_gemm	blocks	vectors: npy, shape=(48071, 256), dtype=float32	28
	cnn_train	activations	images: npy, shape=(4409, 224, 224, 3), dtype=uint8	35
	encoder_infer	data	tokens: npy, shape=(44344, 512), dtype=int32	35
	gpt_train	activations	tokens: npy, shape=(17737, 1025), dtype=int32	55
	lm_finetune (sharded)	params	tokens: npy, shape=(1181, 1025), dtype=int32	63
	loader_bound	input_bound	images: npy, shape=(7681, 64, 64, 3), dtype=uint8	43
	mlp_tabular	params	train: parquet, 480,714 rows, schema=[f0: float, f1: flo...	36
	knn_search (unseen)	blocks	index: npy, shape=(240357, 128), dtype=float32	27
	llm_generate (unseen) (sharded)	kv_cache	prompts: npy, shape=(3313, 2048), dtype=int32	59
	recsys_train (unseen) (sharded)	params	clicks: parquet, 1,786,450 rows, schema=[user: int64, it...	53

B A complete prompt

The prompt every language-model approach reads, here for the init-tmpfs Job of a k8s-2 test task. The init container's memory limit is 281 Mi: 256 Mi plus the 25 MiB it stages onto the memory-backed volume. A Flyte pod's prompt shows the task module in place of `job.py` and the manifest, and reports history peaks with the runtime included.

System

```
You size batch jobs before they run: Flyte tasks, or Kubernetes Jobs given as a Python module and its manifest.
     Given the code, the metadata of this run's inputs, and what earlier runs of the same job used, choose the
     cheapest request that runs it: memory, CPU cores, and a GPU or none. Memory is a hard limit: below the
    peak the container is OOM-killed and retried with double. A Flyte task's memory includes the Flyte runtime
    ; a Kubernetes container's includes whatever the job loads to fetch its inputs, and files written to a
    memory-backed emptyDir (medium: Memory) are charged to it. CPU below the busiest 15 seconds' use throttles
     the task and it finishes late. Thread pools the code does not size get one thread. gpu is the peak GPU
    memory to provision, or none for a task that does not use CUDA; it lands on the cheapest machine that
    holds it -- T4 (15 GiB, $0.53/h), L4 (22 GiB, $0.98/h), A10G (22 GiB, $1.21/h), L40S (44 GiB, $2.24/h), 4
    xL4 (88 GiB, $4.60/h), 8xT4 (120 GiB, $7.82/h), 4xL40S (176 GiB, $10.49/h) -- and a GPU pod comes with its
     host memory and cores. A multi-GPU machine only helps code that spreads its model over every visible GPU.
     A GPU task given no GPU fails at start; a GPU given to a CPU task is paid for the whole run.
```

User

```
### Job code (job.py)
‘‘‘python
import pandas as pd
```

```
import json
import os
import sys


def join_orders_catalog(orders_path: str, catalog_path: str) -> dict:
    orders = pd.read_parquet(orders_path)
    catalog = pd.read_parquet(catalog_path)
    catalog = catalog.drop_duplicates("sku", keep="last")
    joined = orders.merge(catalog, on="sku", how="inner")
    joined["revenue"] = joined["qty"] * joined["price"]
    by_brand = joined.groupby("brand", observed=True)["revenue"].sum()
    return {"rows": int(len(joined)), "brands": int(len(by_brand))}


if __name__ == "__main__":
    result = join_orders_catalog(sys.argv[1], sys.argv[2])
    os.makedirs("/data/out", exist_ok=True)
    with open("/data/out/result.json", "w") as f:
        json.dump(result, f)
```

Job manifest
```yaml
apiVersion: batch/v1
kind: Job
metadata:
  generateName: pandas-join-
  labels: {app.kubernetes.io/name: pandas-join}
spec:
  backoffLimit: 0
  template:
    spec:
      restartPolicy: Never
      initContainers:
        - name: stage-inputs
          image: amazon/aws-cli:2.17.0
          command: [sh, -c, "aws s3 cp $INPUT_ORDERS /data/orders.parquet && aws s3 cp $INPUT_CATALOG /data/
     catalog.parquet"]
          env: [{name: INPUT_ORDERS, value: "s3://batch-data/pandas-join/56ef2668/run-1/orders.parquet"}, {
     name: INPUT_CATALOG, value: "s3://batch-data/pandas-join/56ef2668/run-1/catalog.parquet"}]
          resources: {requests: {cpu: 500m, memory: 256Mi}, limits: {memory: 281Mi}}
          volumeMounts: [{name: data, mountPath: /data}]
      containers:
        - name: main
          image: registry.internal/batch/pandas-join:84b6a081
          command: [sh, -c, "python /app/job.py /data/orders.parquet /data/catalog.parquet && aws s3 cp /data/
     out/result.json $OUTPUT_URI"]
          env:
            - {name: OMP_NUM_THREADS, value: "1"}
            - {name: OPENBLAS_NUM_THREADS, value: "1"}
            - {name: MKL_NUM_THREADS, value: "1"}
            - {name: OUTPUT_URI, value: "s3://batch-data/pandas-join/56ef2668/run-1/out/"}
          resources: {}  # to be decided
          volumeMounts: [{name: data, mountPath: /data}]
      volumes:
        - name: data
          emptyDir: {medium: Memory}
```

This run's inputs
```
orders: parquet, 15.48 MiB, 632,510 rows, schema=[order_id: int64, sku: int64, qty: int32, price: double, ts:
     int64]
catalog: parquet, 9.43 MiB, 80,000 rows, schema=[sku: int64, attr0: double, attr1: double, attr2: double,
     attr3: double, attr4: double, attr5: double, attr6: double, attr7: double, attr8: double, attr9: double,
     attr10: double, attr11: double, brand: string (dictionary)]
```

Earlier runs of this job (container memory peaks, from the cluster's metrics)

```
- inputs orders: parquet, 49.05 MiB, 2,013,227 rows; catalog: parquet, 9.43 MiB, 80,000 rows -> peak memory
    668 MiB, mean 0.99 cores, 1 s

### Options
memory: 256Mi, 384Mi, 512Mi, 768Mi, 1Gi, 1.5Gi, 2Gi, 3Gi, 4Gi, 6Gi, 8Gi, 12Gi, 16Gi, 24Gi, 32Gi, 48Gi, 64Gi
cpu: 0.25, 0.5, 0.75, 1, 1.25, 1.5, 1.75, 2, 2.5, 3, 3.5, 4, 5, 6, 7, 8
gpu: none, 1GiB, 1.5GiB, 2GiB, 3GiB, 4GiB, 6GiB, 8GiB, 10GiB, 12GiB, 15GiB, 22GiB, 32GiB, 44GiB, 64GiB, 88GiB,
     120GiB, 176GiB

Answer with JSON only: {"memory": "<one memory option>", "cpu": "<one cpu option>", "gpu": "<one gpu option>"}
```

C Hyperparameters and tuned settings

Table 17: Settings. Tuned values are seed 1's; where a value is at the edge of the grid it was searched over, the grid is given.

approach	setting	value
static	quantiles (tuned)	memory 0.9 (3 Gi), CPU 0.5 (1.25 cores), from {0.5, 0.75, 0.9, 0.95}
rules	margins (tuned)	memory 1.5, CPU 1.0, GPU 1.0; first-run GPU quantile 0.5
rules-sized	cold start (tuned)	512 MiB + 4× input MiB (512 is the grid's largest base); margin 1.5 with history
trees	quantile models	19 per axis, $\tau = 0.05 \ldots 0.95$; 300 rounds, lr 0.06, 31 leaves; $\theta = 0.3$ (the grid's smallest)
zero-shot	generation	27B Qwen (vLLM), temperature 0; 8,192 tokens free-form, or 256 tokens JSON-schema constrained; 16 concurrent requests
decision	torso	Qwen3-1.7B, LoRA r 16, α 32 on q, k, v, o, gate, up, down; prompt $\leq$ 3,072 tokens
decision	head	shared LayerNorm; per axis W_q, W_k: 2,048 $\rightarrow$ 256
decision	optimization	1 epoch over 27,936 rows (2 epochs over 13,968 for single-pod-type ablations); batch 8×2; AdamW 2×10^{-4} (LoRA) / 10^{-3} (head); 5% warm-up, cosine; the three cross-entropies summed with equal weight
decision	calibration	T = 1.09 / 1.04 / 1.04 (memory / CPU / GPU); seed 2: 1.14 / 1.14 / 1.09; seed 3: 1.04 / 1.09 / 1.09; $\theta = 0.3$
RL	group-normalized REINFORCE	$K = 16$, 6,000 prompts, 1 epoch, batch 8×2, AdamW 3×10^{-5} / 10^{-4}, KL β 0.05, reward cap 20 (fixed, not tuned)
RL	calibration	T = 5.0 / 5.0 / 5.0, the top of the search range $[0.3, 5]$ (seed 2's GPU: 4.13)
pools	w on trees (tuned)	× decision 0.5; × RL 0.6 (seed 3: 0.8); × Job-only 0.6 (seed 2: 0.4); × Flyte-only 0.4; $\theta = 0.3$
gate	signal	kNN, $k = 10$, Euclidean on train-standardized features; $q = 0.95$ of validation
cost	constants	CPU node: 0.02 h per attempt, 8 attempts; GPU node: 0.05 h per attempt, 4 attempts; OOM and delay penalties 1 (in label-request costs); no-GPU failure 0.05 h

D Runs

Every cluster run behind this paper is listed below. They ran in the project `resource-tuner-model-factory` on Union's demo cluster, `https://demo.hosted.unionai.cloud`. The three corpora were built in earlier

rounds: RightSizeMem v2 (`rsb-v2-corpus-2`), RightSizeCPU big-1 (`rsc-big-1`) and RightSizeGPU big-2 (`rsg-big2-1`).

run	what
`rt-k8sprobe-1`	staging probe: boto3 footprint and throughput, tmpfs charging
`rt-k8sval-2`	176 runs as a Flyte task and as `job.py`, as sibling processes in one pod
`rt-k8spod-pilot-1..3`	real-pod probe pilots: LimitRange CPU limits, harness fixes, init-tmpfs failures found
`rt-k8spod-1`	real-pod probe: 100 runs, about 480 pods (Section 7)
`rsk-k8s-1`, `rsk-k8s-1-gate`	the first release (180 MiB runtime, flat init containers), two trainings
`rsk-k8s-2`	k8s-2, seed 1: every approach, the counterfactual and the name test
`rsk-k8s-2-s2`, `-s3`	k8s-2, seeds 2 and 3: every approach except zero-shot
`rt-paper-k2s*`	exports of every request per pod, for the offline analysis
`rt-paper-k2s1-audit`	code audit: which test tasks repeat a training program, and what each family imports
`rt-hub-k8s-2-*`	staging and publishing the release to the Hugging Face Hub (Section 12)

E Supporting tables

Table 18: The difference between a run's Job pod and its Flyte pod, paired on the same runs (Job minus Flyte, in percentage points), for seed 1.

	Job − Flyte regret, paired (pp)	
approach	test	unseen
static	-2.2 [-3.1, -1.3]	+2.2 [+1.2, +3.2]
rules	+1.4 [+0.0, +2.6]	+1.5 [+0.6, +2.5]
rules-sized	+0.9 [-0.3, +1.9]	-2.3 [-3.0, -1.6]
zero-shot 27B	+15.3 [-3.9, +36.9]	-10.5 [-26.8, +6.5]
zero-shot 27B, JSON	+30.3 [+16.3, +47.6]	-7.4 [-38.1, +13.8]
trees	-3.0 [-3.9, -2.2]	-0.2 [-1.2, +0.8]
decision	+1.5 [+0.5, +2.4]	+4.7 [+3.4, +6.0]
decision, argmax	+1.8 [+0.1, +3.7]	+5.3 [+3.5, +6.9]
RL	+0.4 [-0.3, +1.1]	-1.3 [-2.4, -0.2]
RL, argmax	-0.1 [-0.9, +0.9]	-0.0 [-1.4, +1.2]
pool × decision	+1.2 [+0.6, +1.7]	+5.3 [+3.8, +6.7]
pool × decision, argmax	+0.7 [-0.7, +1.9]	+3.8 [+2.1, +5.5]
pool × RL	-0.5 [-1.1, +0.1]	-1.0 [-2.3, +0.2]
pool × RL, argmax	+0.3 [-0.7, +1.3]	+1.0 [-0.4, +2.4]
gate	+0.8 [-0.3, +1.7]	-1.3 [-2.3, -0.2]
decision, Job-only	-7.5 [-8.8, -6.1]	+2.6 [+0.7, +4.7]
decision, Flyte-only	+26.5 [+24.4, +28.8]	+10.1 [+8.0, +12.4]

Table 19: Test pairs of a run's Flyte pod and its Job pod on CPU nodes (3,950 pairs, of which 29.9% have different labels). The first three columns give the share of moved pairs where the request moved the same way as the label; "unmoved, kept" gives the share of unmoved pairs where the request also stayed the same.

	label moved: direction right (%)				
approach	moved	up	down	unmoved, kept	all
rules	24.5	32.3	15.3	90.6	70.9
rules-sized	35.6	48.6	20.4	83.1	68.9
zero-shot 27B	53.8	38.5	69.2	30.4	36.8
trees	60.3	74.8	43.3	78.4	73.0
trees, Job-only	50.4	89.8	4.1	77.0	69.0
trees, Flyte-only	35.2	63.8	1.7	50.6	46.0
decision	63.6	81.2	42.9	78.9	74.3
decision, Job-only	57.3	64.7	48.6	69.2	65.6
decision, Flyte-only	43.3	61.4	21.9	58.8	54.2
RL	56.4	82.6	25.6	74.7	69.2
pool × decision	63.8	81.7	42.9	80.6	75.6
pool × RL, argmax	66.6	80.9	49.7	84.3	79.0
gate	66.1	81.2	48.4	83.8	78.5

Table 20: Regret (%) on the counterfactual's first runs and their real Jobs, with the task's own name and with a neutral one, for seed 1. No model was retrained. These pods exist only inside the counterfactual station, so this table's regret is measured against the label request, as the station scored it.

	test regret (%)			unseen regret (%)		
approach	named	neutral	Δ	named	neutral	Δ
static	83.0	83.0	+0.0	104.2	104.2	+0.0
rules	80.8	80.8	+0.0	82.7	82.7	+0.0
trees	17.9	18.6	+0.7	97.9	98.4	+0.5
decision	10.3	10.2	-0.1	83.7	86.5	+2.8
decision, Job-only	19.4	20.1	+0.7	82.4	83.6	+1.2
decision, Flyte-only	32.6	34.4	+1.8	92.4	93.9	+1.5
RL	11.7	12.3	+0.5	58.1	61.7	+3.6
pool × decision	6.4	7.2	+0.8	86.9	88.2	+1.4
pool × RL, argmax	12.9	13.4	+0.5	109.3	109.8	+0.5

Table 21: Job regret (%) on test CPU-node pods, by staging, for seed 1.

approach	init-disk	init-tmpfs	boto3-disk	boto3-tmpfs
static	97.3	90.0	70.5	80.4
rules	56.4	64.0	51.1	53.3
rules-sized	47.5	51.7	54.5	44.6
zero-shot 27B	55.2	81.2	37.6	93.9
zero-shot 27B, JSON	30.1	73.9	67.8	84.8
trees	19.9	20.7	15.3	12.2
decision	14.3	11.0	14.4	13.0
RL, argmax	12.7	10.1	**10.9**	10.0
pool × decision	**11.6**	**9.0**	12.0	**9.8**
pool × RL, argmax	16.4	13.7	13.2	10.7
gate	16.6	14.4	13.4	11.5

Table 22: Both read-outs of each trained model, Job and Flyte regret (%) for seed 1. Validation (Job and Flyte pods together) chooses the read-out the leaderboard reports.

	Job pods			Flyte pods		
approach	val	test	unseen	test	unseen	chosen
decision	17.8	17.9	51.9	16.4	47.2	✓
decision, argmax	33.9	36.7	74.1	34.9	68.8	
decision, Job-only	18.3	18.5	60.3	25.9	57.7	✓
decision, Job-only, argmax	34.8	35.4	79.5	48.0	84.6	
RL	18.9	19.6	42.8	19.3	44.0	
RL, argmax	14.1	13.5	50.6	13.6	50.6	✓
pool × decision	12.5	11.7	61.0	10.5	55.7	✓
pool × decision, argmax	20.9	20.3	89.2	19.6	85.4	
pool × RL	15.1	15.5	53.5	16.0	54.5	
pool × RL, argmax	11.9	12.5	84.1	12.2	83.1	✓

Table 23: Job regret (%) by corpus, for seed 1.

	test			unseen		
approach	Mem	CPU	GPU	Mem	CPU	GPU
static	52.5	130.2	143.2	67.4	136.1	162.7
rules	50.0	64.1	26.2	80.1	24.8	48.6
rules-sized	43.0	59.5	26.2	50.9	24.5	48.6
trees	20.4	12.7	15.3	34.2	87.3	103.3
decision	15.4	10.6	33.4	42.0	61.4	54.4
RL, argmax	14.3	6.5	21.9	44.9	63.1	46.5
pool × decision	12.1	9.1	14.7	32.3	71.6	81.7
pool × RL, argmax	15.8	10.9	8.1	38.9	93.9	122.3
gate	16.5	10.9	10.8	39.2	36.1	51.6

Table 24: Every approach on the zero-shot model's sample: 130 test and 150 unseen Job pods, seed 1.

approach	test	unseen
static	101.6	134.1
rules	39.1	47.9
rules-sized	34.0	40.4
zero-shot 27B	58.5	45.8
zero-shot 27B, JSON	58.0	47.0
trees	19.2	89.1
decision	17.1	52.1
RL, argmax	12.0	47.1
pool × decision	14.2	72.2
pool × RL, argmax	10.0	98.6
gate	12.2	43.5

Table 25: The staging counterfactual for seed 1, as scored by the counterfactual station. “Moved, right”: among pairs whose label moved when the volume switched from disk to tmpfs, the share where the memory request moved the same way. “Unmoved, kept”: among pairs whose label stayed, the share where the request stayed. “Fetcher moved”: the first measure for a switch from an init container to boto3.

	test			unseen		
approach	moved, right	unmoved, kept	fetcher moved	moved, right	unmoved, kept	fetcher moved
static	0.0	100.0	0.0	0.0	100.0	0.0
rules	0.0	100.0	0.0	0.0	100.0	0.0
trees	74.7	76.6	30.8	78.1	20.6	24.0
decision	73.6	76.6	44.6	72.6	30.6	37.3
decision, Job-only	77.3	79.0	40.0	63.6	30.0	33.8
decision, Flyte-only	21.3	87.5	15.8	31.9	65.8	30.2
RL	72.2	66.8	35.8	71.6	42.6	29.8
pool × decision	77.3	77.4	39.6	77.4	20.3	32.4
pool × RL, argmax	77.6	82.0	46.7	72.7	31.9	32.0